Vishwas K Singh
SME-FSD

Python Training Day 1

Agenda

Fundamental Terms and Concepts
Installing and setting up python
Literals, Variables, and Numeral Systems
Input and Output in Python (Console I/O)
Operators and Expressions

Timings

9.30 to 11.00AM – p1
11 – 11.15 – break
11.15 to 1.30 – p2
1.30 to 2.30 – lunch break
2.30 to 3.45 – p3
3.45 to 4.00 – break
4.00 to 5.30 – p4

Vishwas K Singh
SME-FSD

Have you ever wanted a solution

where you get data from various departments
you have to compile those, generate insights, manualy create a report out of it

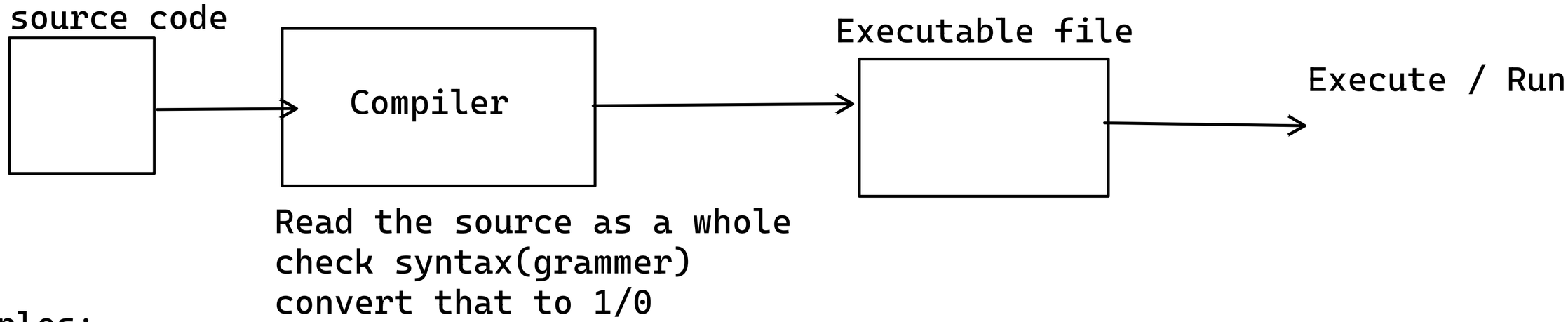
to automate this?

Vishwas K Singh
SME-FSD

Programming language
your High level English kind of language to machine instructions

There are 2 kinds of languages
compiled language
interpreted language

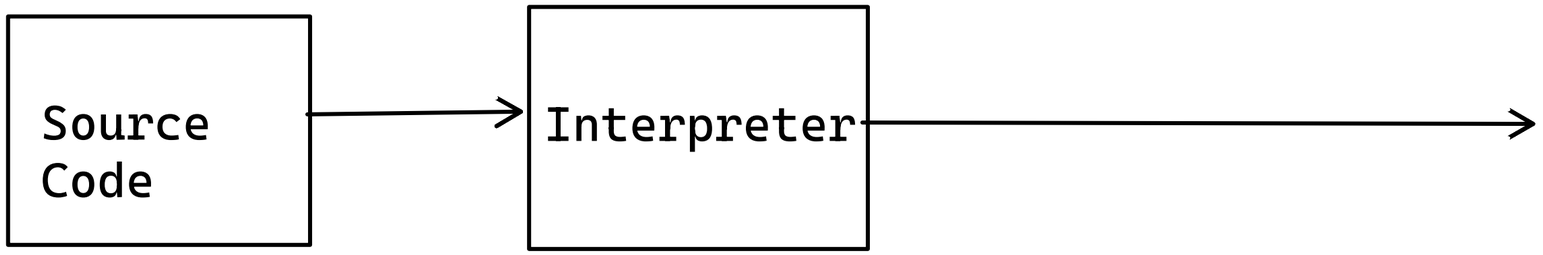
Compiled Language



Read the source as a whole
check syntax(grammer)
convert that to 1/0

Examples:
c, C++, Csharp

Interpreted Language



Examples:
Python
Javascript

Script

Read each command (statement)
Convert that to 1/0
execute the command/ run the command

Python Programming Language — Guido Van Rossum

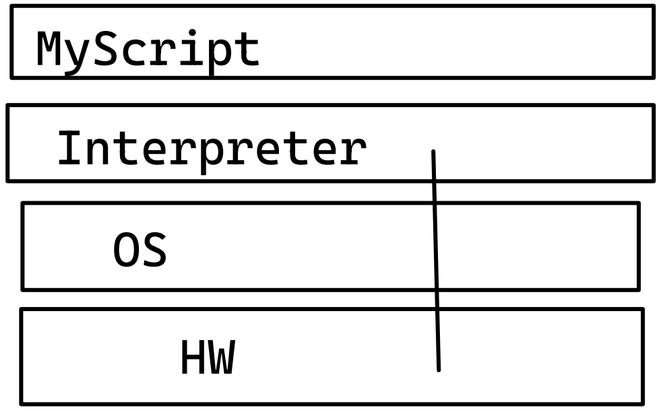
1. General purpose Language
2. Open Source language
3. Architecture Independent Language
4. Dynamically Typed Language (later)

1. I can perform - Web programming, system applications, Numerical Computation

2. Open Source Language:
- Source Code is available, for free to use, modify & redistribute
- Example: Linux (ubuntu, red hat, fedora), Android phone

Architecture Independence

MyScript can be run on any architecture without modifications



Windows 64bit	Windows 64	any other os/ any spec
Intel	AMD	other

Vishwas K Singh
SME-FSD

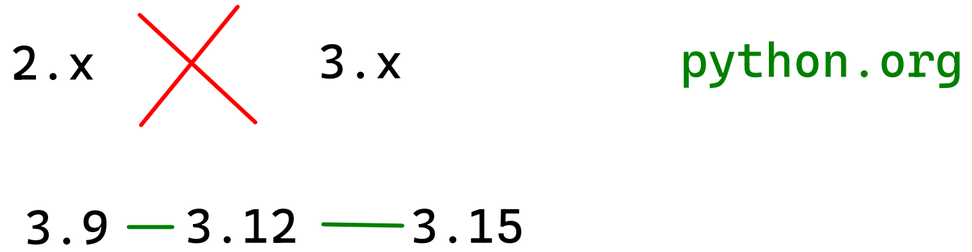
Java / Dotnet vs Python

- Its almost similar
- Python actually takes features from c, c++, Java
- they are Object oriented Language, but python is Functional programming Language

Interpreter of Python
- Needed to run a Python Script

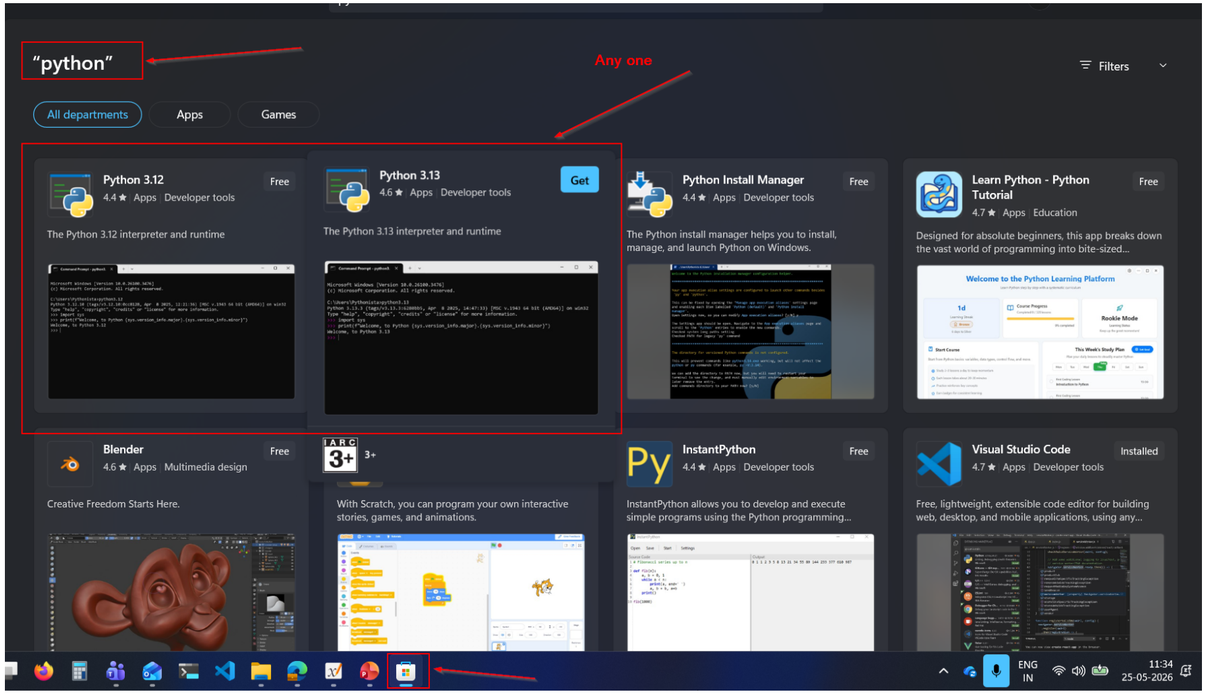
Versions
- C Cython C Code + Python can be used
- Java Jython Java Code + Python
- R RPython R Code + Python
- Python Pypi

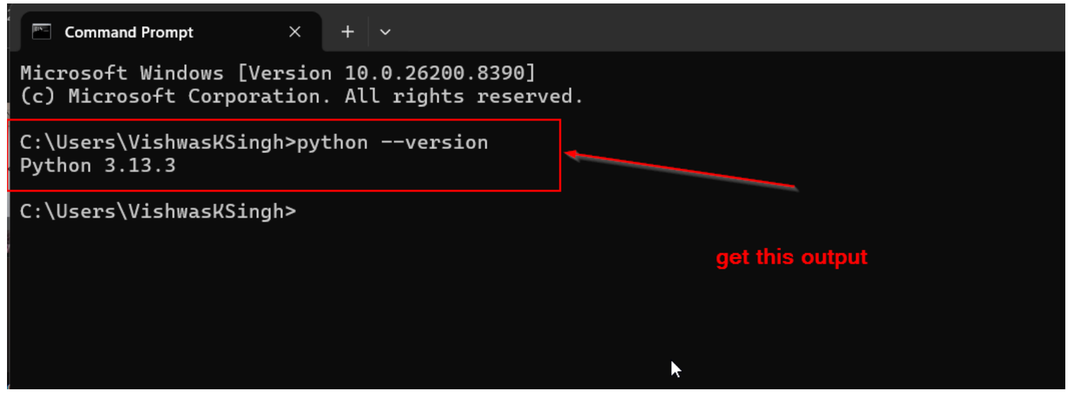
Versions of Python
2.x.x
3.x.x (latest recommended)



Vishwas K Singh
SME-FSD

Lab: Installation of Python

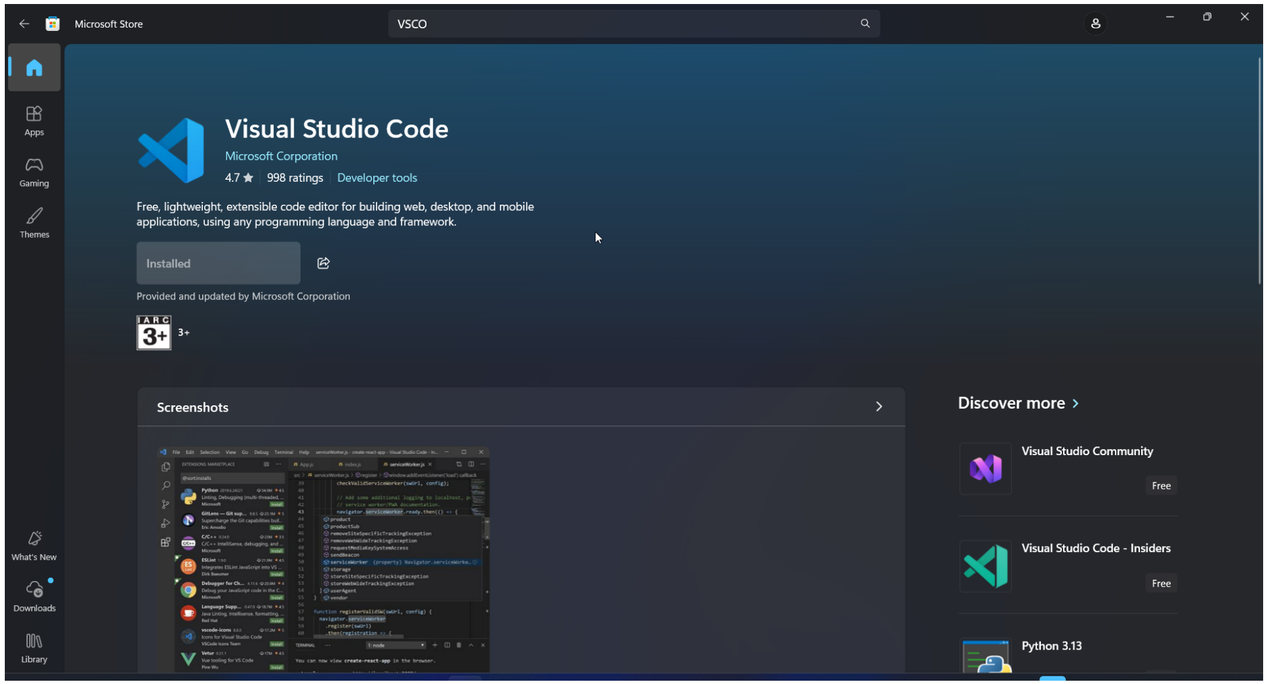




Vishwas K Singh

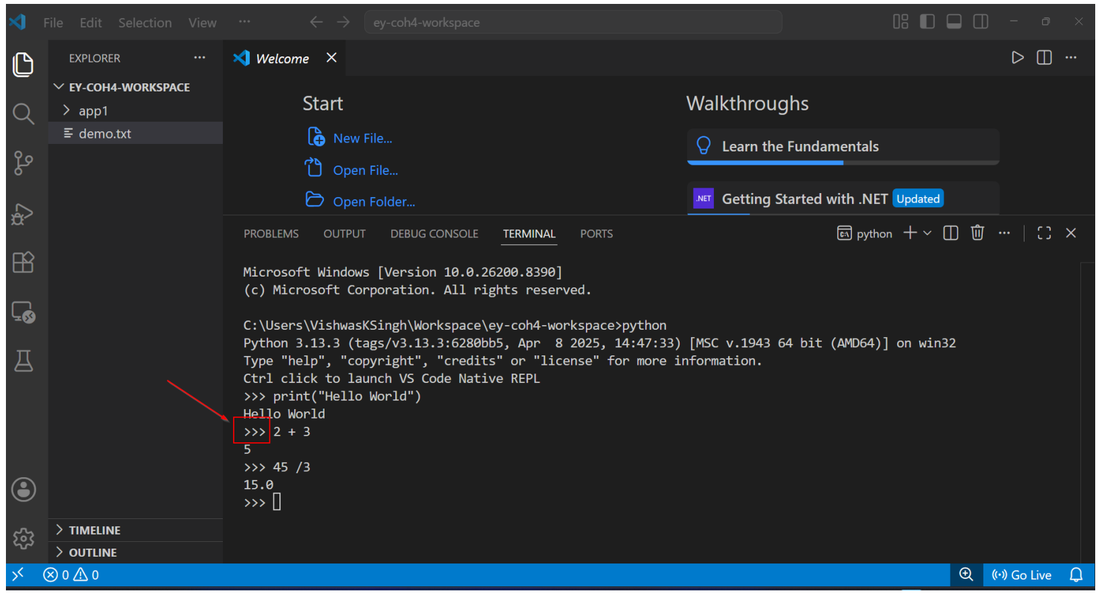
SME-FSD

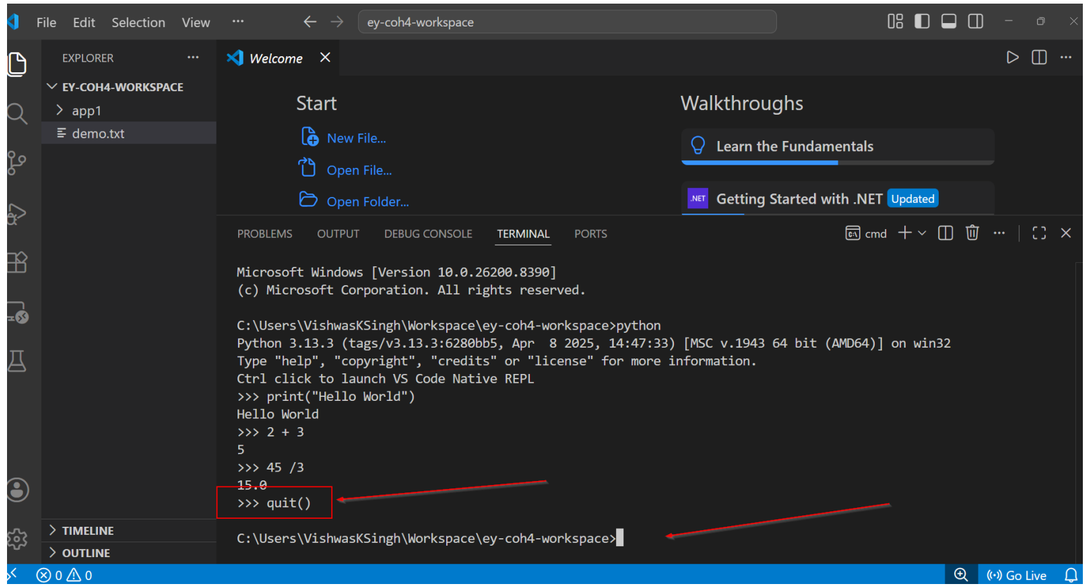
Lab: Text Editor: VS Code



Interacting with Python

1. Interactive Mode / Prompt mode

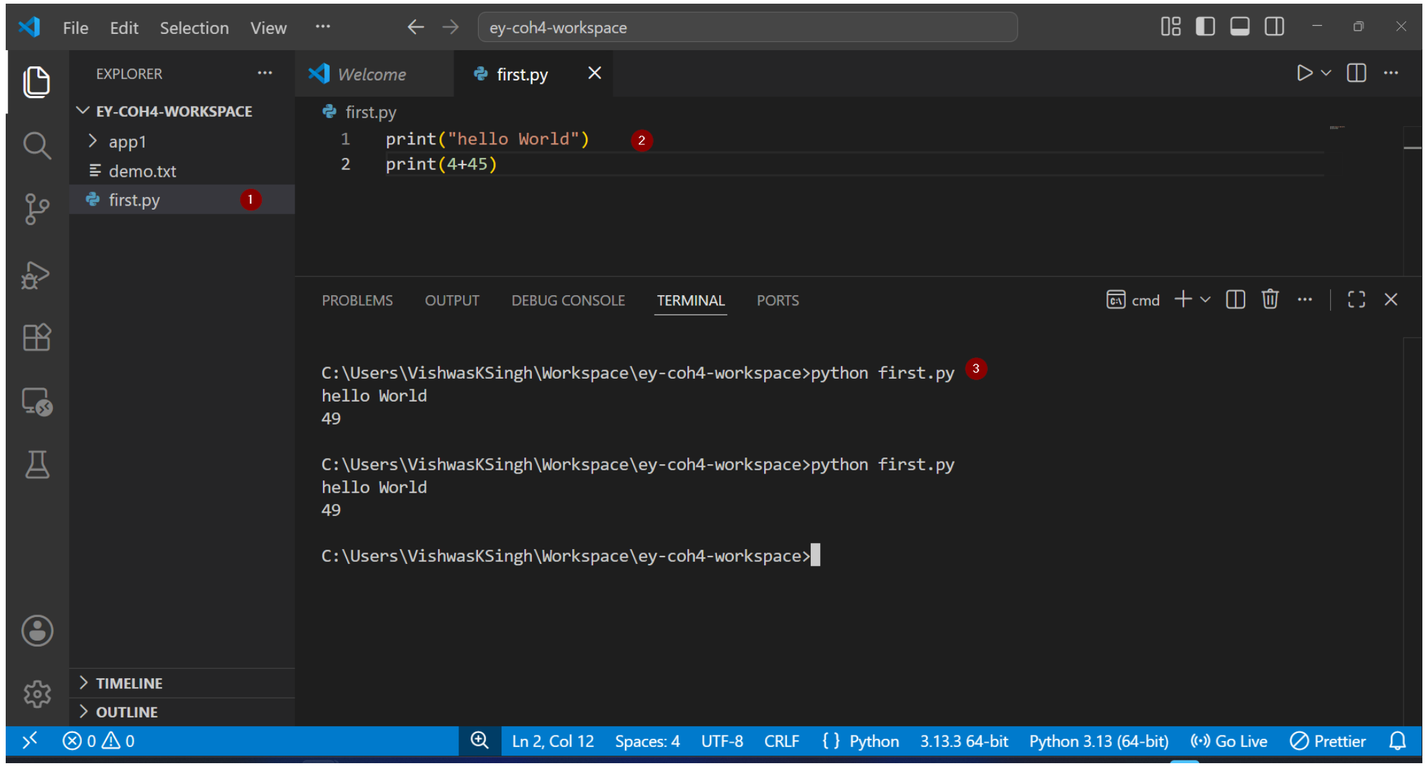




We use this when we are initially creating, or we need to understand working of a small code snippet

Vishwas K Singh
SME-FSD

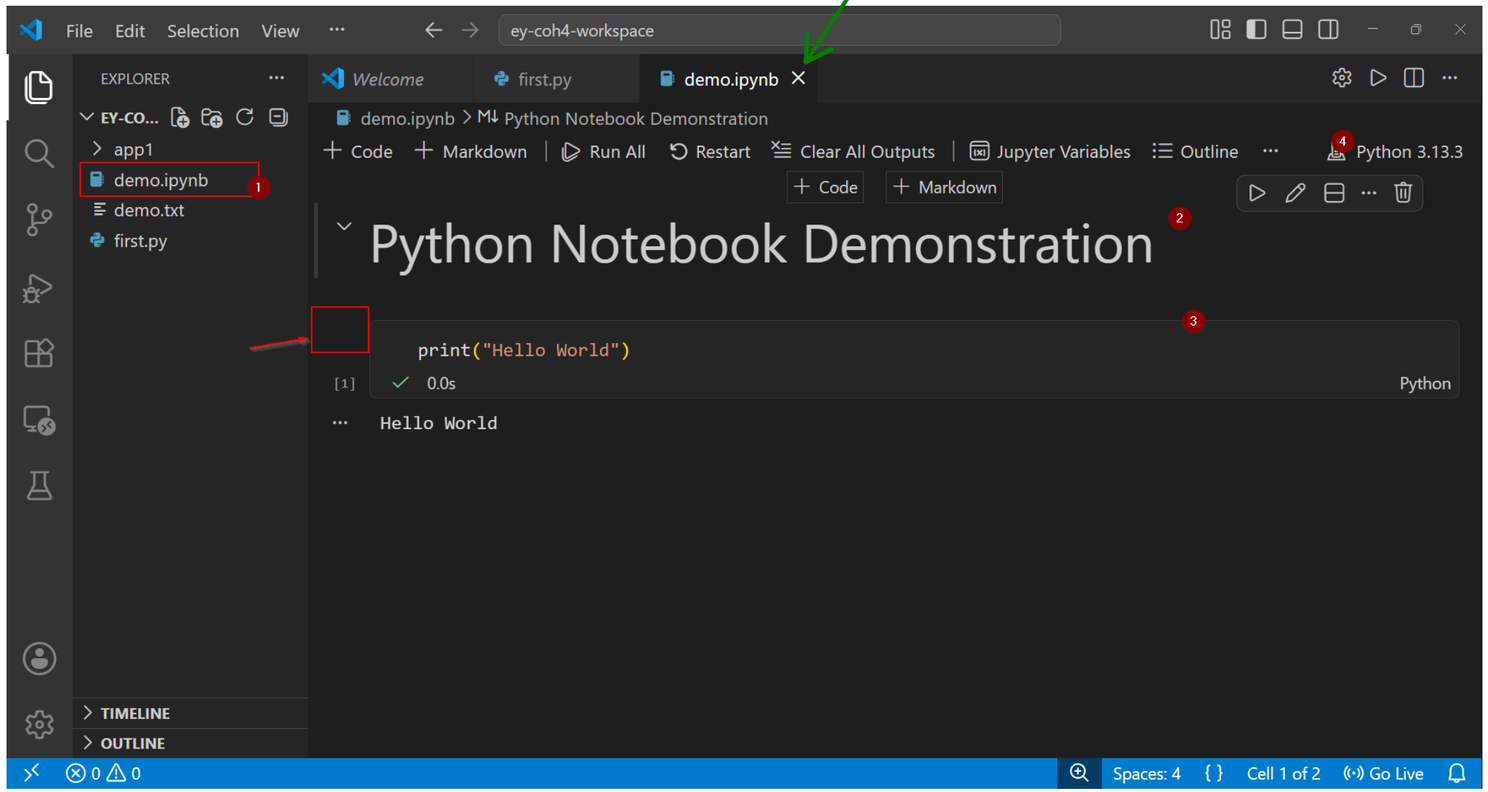
2. Script Mode of Execution



3. Notebook Mode (.ipynb)

Notes + Interactive + saved
Data Analyst/ Scientist

Jupyter Notebooks
Google Colab



Vishwas K Singh
SME-FSD

Python Language Basics

Learning how to program

When ever you are given a problem
Break the problem to sub problem
Find Solution to each sub problem
Integrate it

→

Analyse the sub problem
Input?
Output?
What is relationship b/n Input & Output?
(Also called Logic)

Example:
- get data from various departments
- you have to compile those
- generate insights
- create a report out of it

→

What format is your input → Eg. Excel File
- Read the Excel File(Input)
- Clean It (Logic process)
- Format for next processing (output)

Vishwas K Singh
SME-FSD

To Learn a Programming Language

1. How to store data temporarily
2. How to do Conditions
3. How to repeat (Loops and Functions)
4. What the base paradigm

Python is a object based Functional Programming Language

What are alternatives to Functional Programming Paradigm?
O - Object oriented Programming
P - Procedural Programming Approach

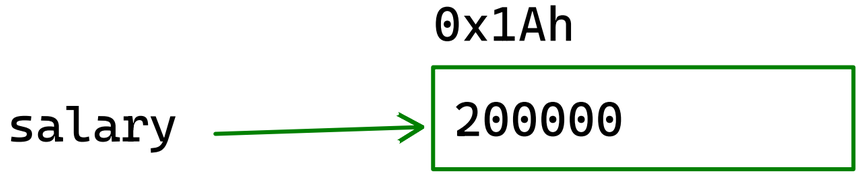
Vishwas K Singh
SME-FSD

1. How to store data temporarily

Variables
Label to memory location

```
salary          =          2000000
```

- salary: variable identifier
- =: Assignment operator
- 2000000: data



- It can contain alphanumeric characters, underscore(_)
- It must always start with a letter or _
- no other special characters

- must not be a keyword
- meaningful (good practice)
- It is case-sensitive

```
C:\Users\VishwasKSingh>python
Python 3.13.3 (tags/v3.13.3:6280b
Type "help", "copyright", "credit
>>> salary = 20000
>>> salary
20000
>>> print(salary)
20000
>>> type(salary)
<class 'int'>
>>> name = "vishwas"
>>> type(name)
<class 'str'>
>>> bonus = 2135.6789
>>> type(bonus)
<class 'float'>
>>>
```

Keywords

```
False
None
True
and
as
assert
async
await
break
class
continue
def
del
elif
else
except
finally
for
from
global
if
import
in
is
lambda
nonlocal
not
or
pass
raise
return
try
while
with
yield
```

Vishwas K Singh
SME-FSD

Data Type
- Int (1234)
- Float (12.34)
- String ("12.34")
- Bool (True)
- Complex (2+3j)

```
>>> k = 1234
>>> k
1234
>>> type(k)
<class 'int'>
>>> k1 = 12_00_234
>>> type(k1)
<class 'int'>
>>> k2 = 0b1010
>>> k2
10
>>> type(k2)
<class 'int'>
>>> k3 = 0x0A
>>> k3
10
>>> type(k3)
<class 'int'>
>>> k4 = 0o27
>>> k4
23
>>> type(k4)
<class 'int'>
>>> k5 = complex(2,3)
>>> k5
(2+3j)
>>> type(k5)
<class 'complex'>
>>>
```

```
>>> f1 = 12.3456
>>> type(f1)
<class 'float'>
>>> f2 = 123456e-4
>>> f2
12.3456
>>> type(f2)
<class 'float'>
>>>
```

```
>>> f3 = True
>>> f3
True
>>> type(f3)
<class 'bool'>
>>> f4 = False
>>> type(f4)
<class 'bool'>
>>> f4
False
>>>
```

```
>>> s1 = "vishwas"
>>> s1
'vishwas'
>>> type(s1)
<class 'str'>
>>> s2 = 'vishwas'
>>> s2
'vishwas'
>>> type(s2)
<class 'str'>
>>> s3 = '''vishwas'''
>>> s3
'vishwas'
>>> type(s3)
<class 'str'>
>>> s4 = """vishwas"""
>>> s4
'vishwas'
>>> type(s4)
<class 'str'>
>>> s5 = "This is Vishwas's book"
>>> s6 = '''
... This is an example of
... multiline value. It is considered
... as a single string itself
... '''
>>> s6
'\nThis is an example of\nmultiline value. It is considered\nas a single string itself\n'
>>>
```

```
>>> s7 = "\nI am Vishwas K Singh\n I am Learning python"
>>> s7
'\nI am Vishwas K Singh\n I am Learning python'
>>> print(s7)

I am Vishwas K Singh
 I am Learning python
>>> s8 = "C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\first.py"
  File "<python-input-65>", line 1
    s8 = "C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\first.py"
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 2-3: truncated \UXXXXXXXX escape
>>> s8 = "C:\\Users\\VishwasKSingh\\Workspace\\ey-coh4-workspace\\first.py"
>>> s8
'C:\\Users\\VishwasKSingh\\Workspace\\ey-coh4-workspace\\first.py'
>>> print(s8)
C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\first.py
>>> s8
'C:\\Users\\VishwasKSingh\\Workspace\\ey-coh4-workspace\\first.py'
>>> s9 = r"C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\first.py"
>>> s9
'C:\\Users\\VishwasKSingh\\Workspace\\ey-coh4-workspace\\first.py'
>>> name = "vishwas"
>>> greet = f"Hey there!, Welcome to session {name}"
>>> greet
'Hey there!, Welcome to session vishwas'
>>> version = 3
>>> greet = f"Hey there!, Welcome to session {name}, I am learning python {version}"
>>> greet
'Hey there!, Welcome to session vishwas, I am learning python 3'
>>> salary = 123.4567890
>>> greet = f"Hey there!, Welcome to session {name}, I am learning python {version}, I am earning a salary of {salary:.2f}"
>>> greet
'Hey there!, Welcome to session vishwas, I am learning python 3, I am earning a salary of 123.46'
>>>
```

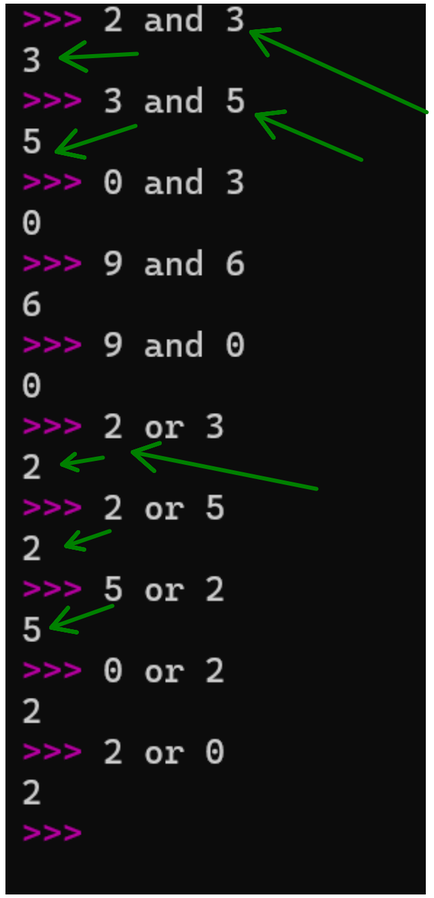
Vishwas K Singh
SME-FSD

Operators

- Assignment (=,+=,-=,*=....)
- Arithmetic (+,-,*,/,%, //(integer division), ** (power))
- Logical (and,or,not)
- Relational (<,>,≤,≥,≠)
- Identity (as)
- Membership (in,not in)
- Bitwise (&,|,~)

Pointers:
- A 0(zero) is always False
- Any non-zero number is always True

For faster evaluation python uses "Short Ciruit Evaluation"
- In a logical expression expr1 and expr2
  expr2 is not evaluated unless expr1(True)
- In a logical expression expr1 or expr2
  expr2 is not evaluated at all when expr1 is already True



Vishwas K Singh
SME-FSD

"Guard Pattern"

```
>>> x = 10
>>> y = 2
>>> x / y
5.0
>>> y = 0
>>> x / y
Traceback (most recent call last):
  File "<python-input-94>", line 1, in <module>
    x / y
    ~~^~~
ZeroDivisionError: division by zero
>>> y != 0 and x / y
False
>>> y = 2
>>> y != 0 and x / y
5.0
>>>
```

Vishwas K Singh
SME-FSD

Lab: GST Calculation v1.0

The GST on Electronic Applicances was reduced from 28% to 18%. If an AC costed Rs.30000(excluding GST) what are the values of GST at both the rates & what is the difference in cost (how much you would save now)?

Assume no other taxes are applied

Vishwas K Singh
SME-FSD

Console I/O in python
- printing to a conole - print(....)
- taking values from a console - input(....)

Input Prompt
i.e message for the user

Pointers:
1. Whatever you take from an input function is always a string
2. You as a developer have to convert to right data type

```
>>> age = input("Enter your age: ")
Enter your age: 56
>>> age
'56'
>>> type(age)
<class 'str'>
>>> age = int(age)
>>> age
56
>>> type(age)
<class 'int'>
>>>
```

Vishwas K Singh
SME-FSD

Lab: GST Calculation v2.0

gst_calcv2.py > ...

```python
# Input Section
ac_cost = float(input("Enter the AC Price: "))
old_gst = float(input("Enter old GST Rate: ")) / 100 # Calculating actual value for 28%
new_gst = float(input("Enter new GST Rate: ")) / 100

# Logic or calculation
old_total_cost = ac_cost + (ac_cost * old_gst)
new_total_cost = ac_cost + (ac_cost * new_gst)

total_savings = old_total_cost - new_total_cost

# Output
print(f"The Cost Saving of a AC at Rs.{ac_cost} with reduction in GST from {old_gst * 100:.2f}% to {new_gst * 100}%\
 is Rs.{total_savings} ")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python gst_calcv2.py
Enter the AC Price: 25000
Enter old GST Rate: 28
Enter new GST Rate: 20
The Cost Saving of a AC at Rs.25000.0 with reduction in GST from 28.00% to 20.0% is Rs.2000.0
```

Vishwas K Singh
SME-FSD

Summary Day 1

We learnt about
1. What is programming How to approach it
2. Different kinds of Programming Language
3. Features of Python Programming Language, different versions, different interpreters
4. Variables, Data Types & Operators, Console I/O in python

What to look out for tomorrow?
- Conditionals
- Loops
- Strings, Lists, Dicts, Tuples, Sets

****

Vishwas K Singh
SME-FSD

Python Training Day 2

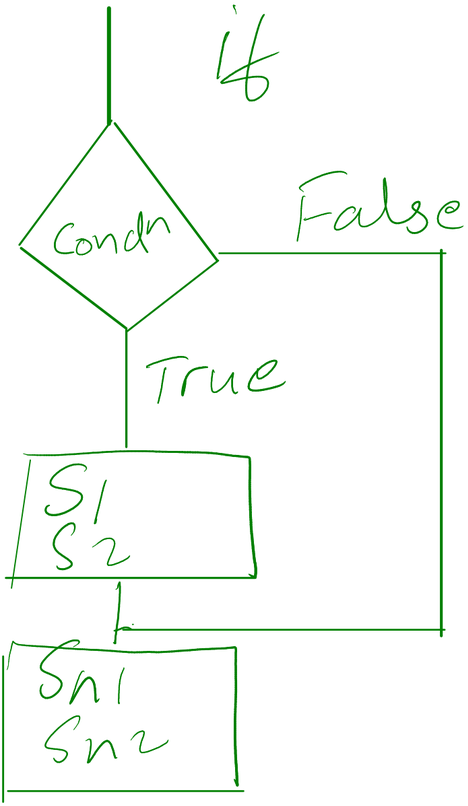
Agenda

- Conditions
- Loops
- Data Structures builtin

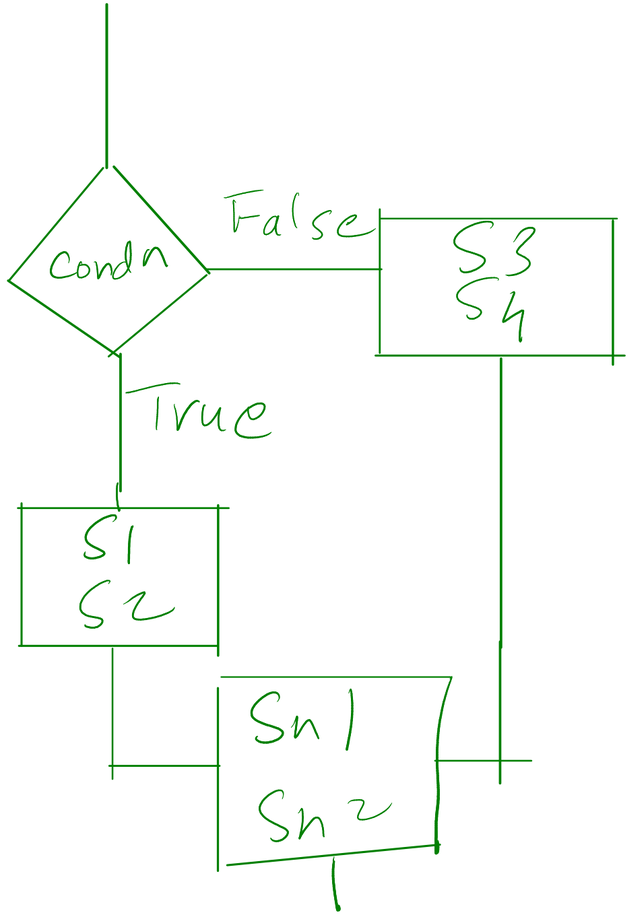
Vishwas K Singh
SME-FSD

if
if-else
if-elif-else
nested if

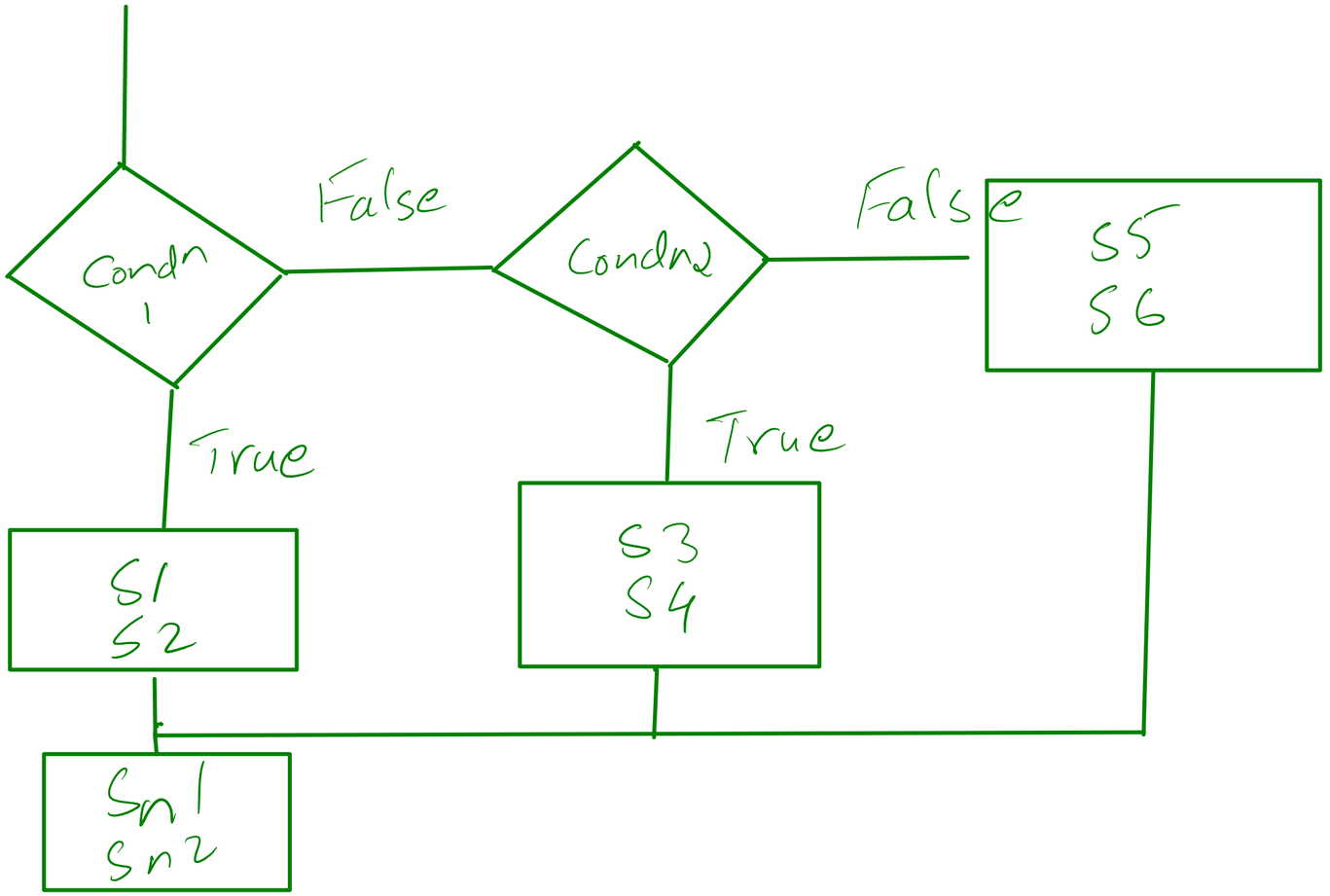
```
if cond:
    s1
    s2
sn1
sn2
```




```
if condn:
    s1
    s2
else:
    s3
    s4
sn1
sn2
```

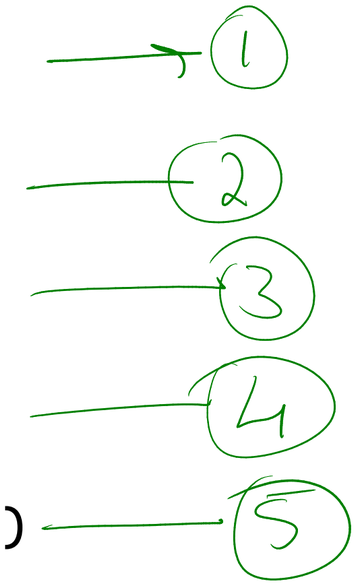


```
if condn1:
    s1
    s2
elif condn2:
    s3
    s4
else:
    s5
    s6
sn1
sn2
```



Vishwas K Singh
SME-FSD

```
x = 100

if x > 50:
     print("x > 50")
elif x > 70:
     print("x > 70")
elif x > 30:
     print("x > 30")
else:
     print("x < 30")

print("I am outside")
```



which of these statments are printed?

1 and 5 only

Vishwas K Singh
SME-FSD

Lab: Build a Investment Calculator, where you invest in MF and you need to see whether SIP or Lumpsum investment is beneficial for the same amount & same time period

Lumpsum: The entire amount is invested on day one and compounds annually for the entire duration.

$$A = P\left(1 + \frac{r}{100}\right)^n$$

SIP: The total amount is divided by the total number of months. That smaller monthly amount is invested at the start of every month, with each installment compounding for its remaining time.

$$M = \frac{P}{n \times 12}$$

$$A = M \times \left[\frac{(1+i)^k - 1}{i}\right] \times (1+i)$$

(Where i is the monthly interest rate $\frac{r}{12\times100}$ and k is the total number of months).

```python
lumpsum_value = principal * (1 + (rate_of_interest)) ** num_years
print(f"The lumpsum invested value is: {lumpsum_value:.2f}")
total_months = num_years * 12
monthly_amount = principal / total_months
print(f"Monthly Amount: {monthly_amount:.2f}")
monthly_interest_rate = rate_of_interest / 12

sip_value = monthly_amount * ((((1+monthly_interest_rate)**total_months)-1)/monthly_interest_rate)*(1+monthly_interest_rate)
print(f"The SIP invested value is: {sip_value:.2f}")

if lumpsum_value > sip_value:
    print("Lumpsum investing is beneficial")
else:
    print("SIP is beneficial")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Enter the rate of interest in percentage: 5
Input number of Years: 3
The lumpsum invested value is: 347287.50
Monthly Amount: 8333.33
The SIP invested value is: 324290.06
Lumpsum investing is beneficial
```

Loops
1. For
2. While Loop

```
for(int i = 0; i< 10; i++){
    print(i)
}
```

c/js/java approach

```
for i in range(10):
    print(i)
```

```
range(stop) -> range object
range(start, stop[, step]) -> range object

Return an object that produces a sequence of integers from start (inclusive)
to stop (exclusive) by step.  range(i, j) produces i, i+1, i+2, ..., j-1.
start defaults to 0, and stop is omitted!  range(4) produces 0, 1, 2, 3.
These are exactly the valid indices for a list of 4 elements.
When step is given, it specifies the increment (or decrement).
```

Vishwas K Singh
SME-FSD

Lab: In the investment calculator, show how the amount (lumpsum & sip) grows year-on-year

investment_calcv2.py

```
19
20      print(f"{year:<6}Rs.{opening_balance:<17.2f}Rs.{interest:<17.2f}Rs.
21      lumpsum_balance = closing
22
23
24  # sip Growth
25  print("\n------SIP INVESTMENT GROWTH -------\n")
26  print(f"Monthly Contribution: {monthly_sip_amount}")
27  print(f"{'Year':<6}{'Total Invested':<18}{'Interest Earned':<18}{'Futur
28  print('-'*60)
29
30  sip_balance = 0
31  total_sip_invested = 0
32
33  for year in range(1, num_years + 1):
34      yearly_interest_earned = 0
35
36      for month in range(1,13):
37          sip_balance += monthly_sip_amount
38          total_sip_invested += monthly_sip_amount
39
40          # Calculate monthly interest (compounded monthly)
41          monthly_interest = sip_balance * monthly_rate_decimal
42          sip_balance += monthly_interest
43
44          yearly_interest_earned += monthly_interest
45
46      print(f"{year:<6}Rs{total_sip_invested:<17.2f}Rs.{yearly_interest_e
47
```

PROBLEMS OUTPUT TERMINAL cmd

```
(c) Microsoft Corporation. All rights reserved.

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python investment_c
alcv2.py
Enter the principal amount: 300000
Enter the rate of interest in percentage (p.a): 5
Input number of Years: 5

------LUMPSUM INVESTMENT GROWTH -------

Starting Principal: 300000.0
Year  Opening Balance   Interest Earned   Closing Balance
------------------------------------------------------------
1     Rs.300000.00      Rs.15000.00       Rs.315000.00
2     Rs.315000.00      Rs.15750.00       Rs.330750.00
3     Rs.330750.00      Rs.16537.50       Rs.347287.50
4     Rs.347287.50      Rs.17364.38       Rs.364651.88
5     Rs.364651.88      Rs.18232.59       Rs.382884.47

------SIP INVESTMENT GROWTH -------

Monthly Contribution: 5000.0
Year  Total Invested    Interest Earned   Future Value
------------------------------------------------------------
1     Rs60000.00        Rs.1650.09        Rs.61650.09
2     Rs120000.00       Rs.4804.22        Rs.126454.31
3     Rs180000.00       Rs.8119.73        Rs.194574.04
4     Rs240000.00       Rs.11604.86       Rs.266178.90
5     Rs300000.00       Rs.15268.30       Rs.341447.21

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>
```

Vishwas K Singh
SME-FSD

While Loop

initial value
Condition
tracking

```
x = 10
while x ≥ 10:
    print(i)
    i += 1
```

Infinite Loops

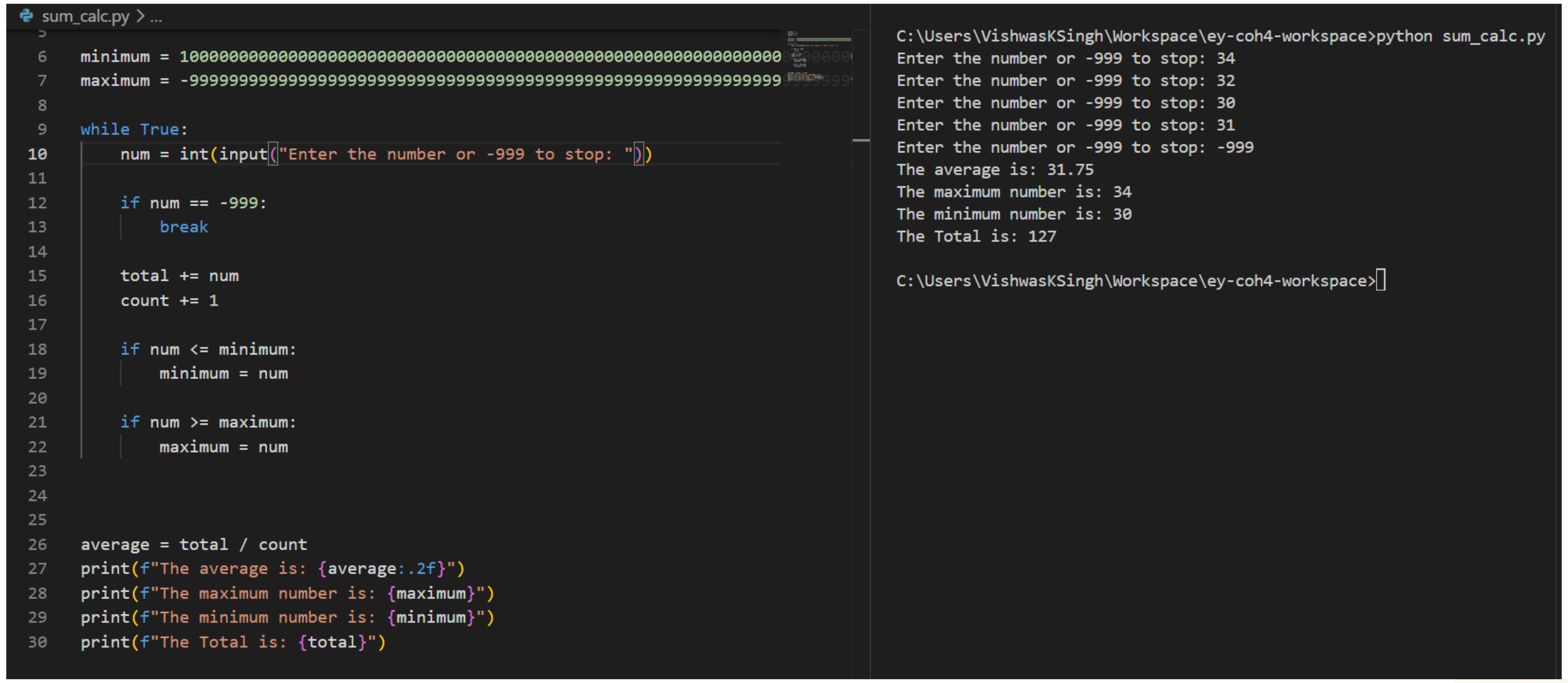
```
while True:

while 234:

while -23:
```

For & while

use the for loop when you know number of iterations

use a while loop when you dont know number of iterations but you know exit condition

Write a script that takes n numbers from the user till he enters -999, find the
sum, average minimum & maximum numbers from the entered numbers



Vishwas K Singh
SME-FSD

String
- class str
- they are immutable (cant be modified)
- Sequence of characters & character can be accessed by index

Poniters:
1. By Default, Always from left to right
2. I can use step value (default 1)
3. If the step value is -ve, it is from left to right

```
>>> name[9:9]
''
>>> name[9:45]
' singh'
>>> name[:]
'vishwas k singh'
>>> name[::2]
'vswsksnh'
>>> name[::1]
'vishwas k singh'
>>> name[::3]
'vhs n'
>>> name[2::2]
'swsksnh'
>>> name[4::2]
'wsksnh'
>>> name[3::2]
'ha  ig'
>>> name[::-1]
'hgnis k sawhsiv'
>>> name[::-2]
'hnskswsv'
>>> name[::-3]
'hikas'
>>> name[::-1]
'hgnis k sawhsiv'
>>> name[-9:-12:-1]
'saw'
>>>
```

```
>>> name = "vishwas k singh"
>>> len(name)
15
>>> name[1]
'i'
>>> name[2]
's'
>>> name[6]
's'
>>> name[-1]
'h'
>>> name[--14]
'h'
>>> name[-12]
'h'
>>> name[-9]
's'
>>>
```

Vishwas K Singh
SME-FSD

Scenario 1:

You have a financial article published on the internet and you want to extract the prices of commodities

Scenario 2:

You have a file with emails received & their email content, You have to categorize them into spam & not spam using email ids

Extract email Ids who have messaged n last 4 days

Scenario 3:

Log File analyze it for suspicious activity like Failed login attempts

Vishwas K Singh
SME-FSD

Lab: You have a log system which ouputs fixed strings, your job is to extract Time Stamp, Level, Message from the text

Example:

"2026-05-26 15:24:10 ERROR Database connection failed."

```
timestamp=log[:19]
level=log[20:25]
message=log[26:]
```

Lab: You are given a File Path, Find out the file extension (if provided)
C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\README.md

extension is md

Lists:

- Stores multiple values
- Different Data Types
- It is mutable
- Indexing & Slicing operations can be applied
- preserves insertion order

```
>>> l1 = []
>>> l2 = list()
>>> l3 = [23,45,78.96,"vishwas",[99,100],{1:"vishwas"},("apple","mango")]
>>> l3[0]
23
>>> l3[2:]
[78.96, 'vishwas', [99, 100], {1: 'vishwas'}, ('apple', 'mango')]
>>> l3[:-4]
[23, 45, 78.96]
>>> dir(list)
['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq_
_', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__iadd__', '__imul
__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__redu
ce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '
__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
>>> l3.append("Grapes")
>>> l3
[23, 45, 78.96, 'vishwas', [99, 100], {1: 'vishwas'}, ('apple', 'mango'), 'Grapes']
>>> l4 = l3.copyt()
Traceback (most recent call last):
  File "<python-input-59>", line 1, in <module>
    l4 = l3.copyt()
         ^^^^^^^^
AttributeError: 'list' object has no attribute 'copyt'. Did you mean: 'copy'?
>>> l4 = l3.copy()
>>> l4
[23, 45, 78.96, 'vishwas', [99, 100], {1: 'vishwas'}, ('apple', 'mango'), 'Grapes']
>>> l3.clear()
>>> l3
[]
```

```
>>> l3.append(1)
>>> l3
[1, 1, 2, 4, 9, 1]
>>> l3.count(1)
3
>>> l4
[23, 45, 78.96, 'vishwas', [99, 100], {1: 'vishwas'}, ('apple', 'mango'), 'Grapes']
>>> l4.extend(l3)
>>> l4
[23, 45, 78.96, 'vishwas', [99, 100], {1: 'vishwas'}, ('apple', 'mango'), 'Grapes', 1, 1, 2, 4, 9, 1]
>>> l5 = l3 + l4
>>> l5
[1, 1, 2, 4, 9, 1, 23, 45, 78.96, 'vishwas', [99, 100], {1: 'vishwas'}, ('apple', 'mango'), 'Grapes', 1, 1, 2, 4, 9, 1]
>>> l5.index(9)
4
>>> l5.index(8)
Traceback (most recent call last):
  File "<python-input-80>", line 1, in <module>
    l5.index(8)
    ~~~~~~~~^^^
ValueError: 8 is not in list
>>> l5.index(2)
2
>>> l5.index(2, 6:)
  File "<python-input-82>", line 1
    l5.index(2, 6:)
                 ^
SyntaxError: invalid syntax
>>> l5.index(2, 6,)
16
>>>
```

Vishwas K Singh
SME-FSD

```
>>> l5.insert(2, 'papaya')
>>> l5
[1, 1, 'papaya', 2, 4, 9, 1, 23, 45, 78.96, 'vishwas', [99, 100], {1: 'vishwas'}, ('apple', 'mango'), 'Grapes', 1, 1, 2, 4, 9, 1]
>>> l5.pop()
1
>>> l5
[1, 1, 'papaya', 2, 4, 9, 1, 23, 45, 78.96, 'vishwas', [99, 100], {1: 'vishwas'}, ('apple', 'mango'), 'Grapes', 1, 1, 2, 4, 9]
>>> l5.remove('papaya')
>>> l5
[1, 1, 2, 4, 9, 1, 23, 45, 78.96, 'vishwas', [99, 100], {1: 'vishwas'}, ('apple', 'mango'), 'Grapes', 1, 1, 2, 4, 9]
>>> l4
[23, 45, 78.96, 'vishwas', [99, 100], {1: 'vishwas'}, ('apple', 'mango'), 'Grapes', 1, 1, 2, 4, 9, 1]
>>> l3
[1, 1, 2, 4, 9, 1]
>>> l3.reverse()
>>> l3
[1, 9, 4, 2, 1, 1]
>>> l3.sort()
>>> l3
[1, 1, 1, 2, 4, 9]
```

Lab: Given a list of IPv4 Addresses and a list of blocked IPs
Find out if blocked IP is present in the list of IPs & set out an alert message

```
incoming_traffic = [
"192.168.1.50",
"185.220.101.5",
"192.168.1.50",
"45.137.22.10"
]
```

```
ip_blacklist = [
"185.220.101.50","45.137.22.10"
]
```

Vishwas K Singh
SME-FSD

Dictionary

- Key: Value Pairs
- key must always be unique
- int & strs
- Values & Keys can be any data type

```
>>> d1 = {}
>>> d1 = dict()
>>> d3 = {23:"mango",45:"apple", "name":"Vishwas", "scores" : [23,45,67,89]}
>>> d3[23]
'mango'
>>> d3[45]
'apple'
>>> d3["name"]
'Vishwas'
>>> d3[78]
Traceback (most recent call last):
  File "<python-input-102>", line 1, in <module>
    d3[78]
    ~~^^^^
KeyError: 78
>>> dir(dict)
['__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__',
 '__getitem__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__ior__', '__iter__', '__le__', '__len__', '__lt__', '__ne__', '__ne
w__', '__or__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__ror__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook
__', 'clear', 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']
>>> d3.get(23)
'mango'
>>> d3.get(78)
>>> d3.get(78,'NA')
'NA'
>>> d3.get(23,'NA')
'mango'
>>> d3.keys()
dict_keys([23, 45, 'name', 'scores'])
>>> d3.values()
dict_values(['mango', 'apple', 'Vishwas', [23, 45, 67, 89]])
>>> d3.items()
dict_items([(23, 'mango'), (45, 'apple'), ('name', 'Vishwas'), ('scores', [23, 45, 67, 89])])
>>> for k,v in d3.items():
...     print(k,':',v,end="|")
...
>>>  mango|45 : apple|name : Vishwas|scores : [23, 45, 67, 89]|
```

```
>>> d3.pop(45)
'apple'
>>> d3
{23: 'mango', 'name': 'Vishwas', 'scores': [23, 45, 67, 89]}
>>> d3.popitem()
('scores', [23, 45, 67, 89])
>>> d3
{23: 'mango', 'name': 'Vishwas'}
>>>
```

```
>>> l1 = ["mango","grapes","papaya"]
>>> inventory = dict.fromkeys(l1,0)
>>> inventory
{'mango': 0, 'grapes': 0, 'papaya': 0}
>>> l2 = [("mango",23),("apple",25),("kiwi",1)]
>>> inventory.update(l2)
>>> inventory
{'mango': 23, 'grapes': 0, 'papaya': 0, 'apple': 25, 'kiwi': 1}
>>> inventory.setdefault("banana",32)
32
>>> inventory
{'mango': 23, 'grapes': 0, 'papaya': 0, 'apple': 25, 'kiwi': 1, 'banana': 32}
>>> inventory.setdefault("mango",32)
23
>>> inventory
{'mango': 23, 'grapes': 0, 'papaya': 0, 'apple': 25, 'kiwi': 1, 'banana': 32}
>>>
```

Vishwas K Singh

SME-FSD

Tuples:

- Similar to list
- Immutable
- indexing & slicing is possible
- support compression & extraction
- used internally by python
- index, count

```
>>> t1 = (23,45,67)
>>> t1[0]
23
>>> t1[:2]
(23, 45)
>>> t1
(23, 45, 67)
>>> t2 = 23,45,98
>>> t2
(23, 45, 98)
>>> a,b,c = t2
>>> a
23
>>> b
45
>>> c
98
>>> t2 > t1
True
>>> t2.index(98)
2
>>> t3 = 1,2,3,4,56,7,1,2,4,5
>>> t3
(1, 2, 3, 4, 56, 7, 1, 2, 4, 5)
>>> t3.count(1)
2
>>> dir(tuple)
['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__
getitem__', '__getnewargs__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '
__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'count', 'index']
```

Vishwas K Singh

SME-FSD

Set

- it is not subscriptable (indexing/slicing not possible)
- Unique
- Operations on sets (mathematical)
  - Union, Intersection, superset, subset,Add, Difference

```
>>> s1 = {1,2,3,1,2,3,1,33,1,5,6}
>>> s1
{1, 2, 3, 33, 5, 6}
>>> s2 = {99,1,2,1,2,23,99,68,75,1,2}
>>> s2
{1, 2, 99, 68, 23, 75}
>>> dir(set)
['__and__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__
getstate__', '__gt__', '__hash__', '__iand__', '__init__', '__init_subclass__', '__ior__', '__isub__', '__iter__', '__ixor__', '__le__', '__len__', '__lt__'
, '__ne__', '__new__', '__or__', '__rand__', '__reduce__', '__reduce_ex__', '__repr__', '__ror__', '__rsub__', '__rxor__', '__setattr__', '__sizeof__', '__s
tr__', '__sub__', '__subclasshook__', '__xor__', 'add', 'clear', 'copy', 'difference', 'difference_update', 'discard', 'intersection', 'intersection_update'
, 'isdisjoint', 'issubset', 'issuperset', 'pop', 'remove', 'symmetric_difference', 'symmetric_difference_update', 'union', 'update']
>>> help(set.add)
Help on method_descriptor:

add(self, object, /) unbound builtins.set method
    Add an element to a set.

    This has no effect if the element is already present.

>>> s2.add(678)
>>> s2
{1, 2, 99, 68, 678, 23, 75}
>>> s3 = s1 - s2
>>> s3
{33, 3, 5, 6}
>>> s4 = s1.difference(s2)
>>> s4
{33, 3, 5, 6}
>>> s5 = s1.intersection(s2)
>>> s5
{1, 2}
>>> s1.issuperset(s2)
False
>>> s1.issubset(s2)
False
>>> s6 = s1.union(s2)
>>> s6
{1, 2, 3, 33, 5, 6, 99, 68, 678, 75, 23}
>>>
```

Vishwas K Singh
SME-FSD

Lab: Frequency Counter

Given a string calculate frequencies of each character

Example:

"vishwassingh"

"v":1,"i":2,"s":3,"h":2.....

freq_counter.py > ...

```python

inp_str = input("Enter input string: ")

char_counts = {}
for chr in inp_str:
    char_counts[chr] = char_counts.get(chr, 0) + 1

print(f"Frequencies: \n{char_counts}")
```

```
C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python freq_counter.py
Enter input string: vishwassingh
Frequencies:
{'v': 1, 'i': 2, 's': 3, 'h': 2, 'w': 1, 'a': 1, 'n': 1, 'g': 1}

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>
```

Vishwas K Singh
SME-FSD

Summary of Day 2

1. Conditions - if,if-else,if-elif-else,nested if
2. loops - for, while, range
3. Builtin DS - Strings, Lists, Dict, Tuples Sets

What can we expect tomorrow:

- Functions, Modules, packages
- OOPs Basics
- File & Exception Handling
- Builtin & Custom Modules

****

Vishwas K Singh
SME-FSD

Python Programming Day 3

Agenda

- Functions, Modules, packages
- OOPs Basics
- File & Exception Handling
- Builtin & Custom Modules

Vishwas K Singh
SME-FSD

Functions

- First class citizens
- they can be passed as variables
- Gives different outputs for different inputs

```python
def display_menu():
    print("\n-------Calculator----------\n1.Add\n2.Subtract\n3.Multiply\n4.Divi

def add(a,b):
    return a+b

def sub(a,b):
    return a-b

def mul(a,b):
    return a*b

def div(a,b):
    return a/b


display_menu()
choice = int(input("Enter the choice: "))
a = float(input("Enter first Number: "))
b = float(input("Enter second Number: "))

if choice == 1:
    print(f"{a}+{b} = {add(a,b)}")
elif choice == 2:
    print(f"{a}-{b} = {sub(a,b)}")
elif choice == 3:
    print(f"{a}*{b} = {mul(a,b)}")
elif choice == 4:
    print(f"{a}/{b} = {div(a,b)}")
```

Vishwas K Singh
SME-FSD

Local & Global Variables

```python
def display_menu():
    print("\n-------Calculator-----------\n1.Add\n2.Subtract\n3.Multiply\n4.Divi
def add(a,b):
    return a+b

def sub(a,b):
    return a-b

def mul(a,b):
    return a*b

def div(a,b):
    return a/b


display_menu()
choice = int(input("Enter the choice: "))
a = float(input("Enter first Number: "))
b = float(input("Enter second Number: "))

if choice == 1:
    print(f"{a}+{b} = {add(a,b)}")
elif choice == 2:
    print(f"{a}-{b} = {sub(a,b)}")
elif choice == 3:
    print(f"{a}*{b} = {mul(a,b)}")
elif choice == 4:
    print(f"{a}/{b} = {div(a,b)}")
```

Local Variables

Global Variables

Vishwas K Singh
SME-FSD

When we define a variable with same name as global variable, by default it is considered as a local variable

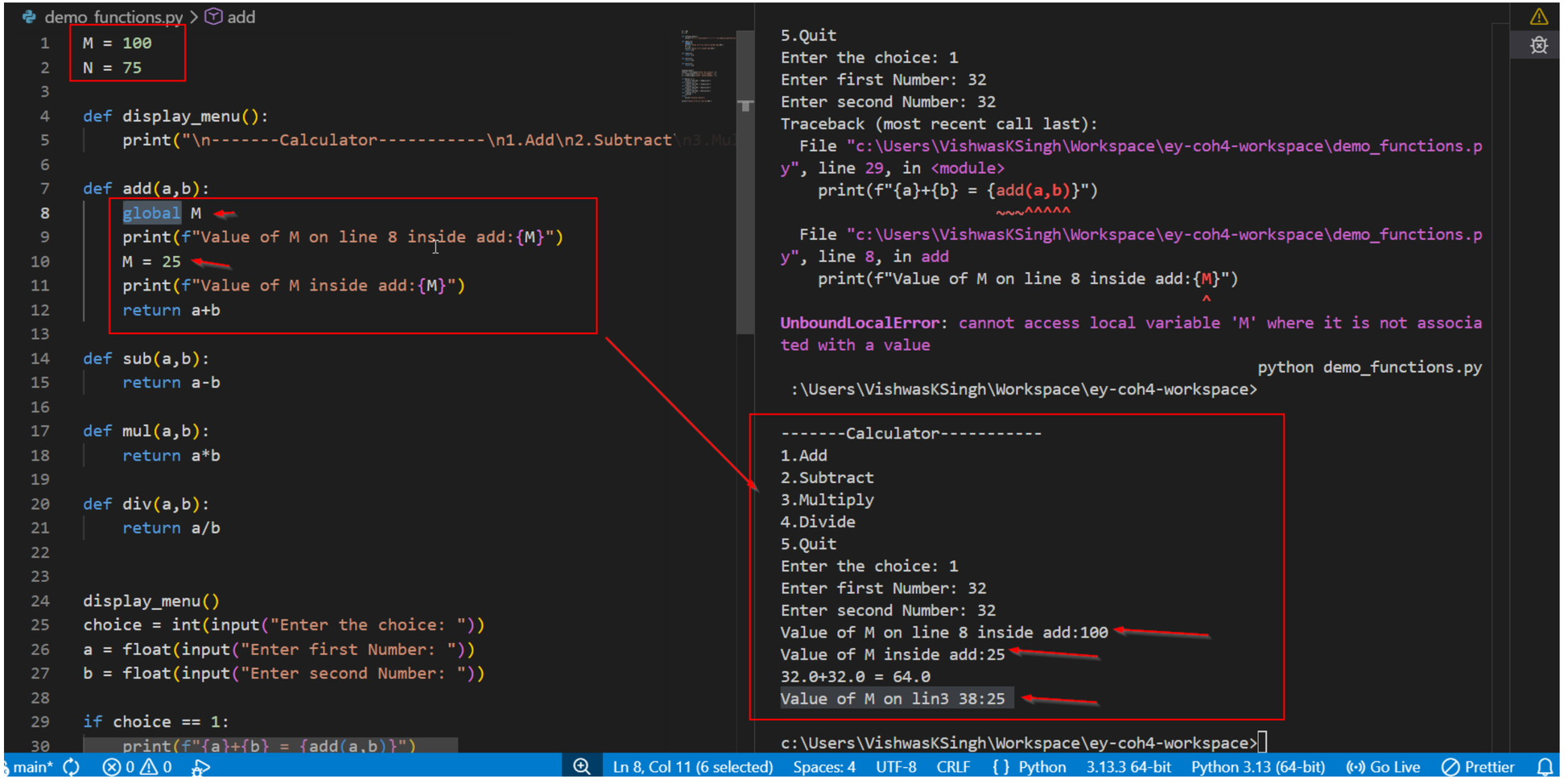
demo_functions.py

```python
M = 100
N = 75

def display_menu():
    print("\n-------Calculator-----------\n1.Add\n2.Subtract\n3.Mul

def add(a,b):
    print(f"Value of M on line 8 inside add:{M}")
    M = 25
    print(f"Value of M inside add:{M}")
    return a+b

def sub(a,b):
    return a-b

def mul(a,b):
    return a*b

def div(a,b):
    return a/b


display_menu()
choice = int(input("Enter the choice: "))
a = float(input("Enter first Number: "))
b = float(input("Enter second Number: "))

if choice == 1:
    print(f"{a}+{b} = {add(a,b)}")
elif choice == 2:
```

```
4.Divide
5.Quit
Enter the choice: 1
Enter first Number: 32
Enter second Number: 32
Value of M on line 8 inside add:100
32.0+32.0 = 64.0
Value of M on lin3 38:100

c:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python demo_functions.py

-------Calculator-----------
1.Add
2.Subtract
3.Multiply
4.Divide
5.Quit
Enter the choice: 1
Enter first Number: 32
Enter second Number: 32
Traceback (most recent call last):
  File "c:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\demo_functions.p
y", line 29, in <module>
    print(f"{a}+{b} = {add(a,b)}")
                       ~~~^^^^^
  File "c:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\demo_functions.p
y", line 8, in add
    print(f"Value of M on line 8 inside add:{M}")
                                             ^
UnboundLocalError: cannot access local variable 'M' where it is not associa
ted with a value

c:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>
```

Vishwas K Singh
SME-FSD

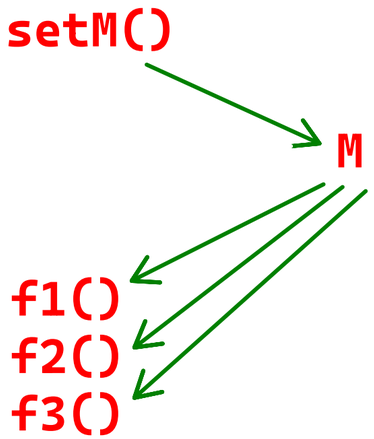
To consider a variable as global we use `global <var_name>` and we are able to modify its value



Though there is a way, but not recommended

Vishwas K Singh
SME-FSD

When to use Global key word



Readability of Functions in module

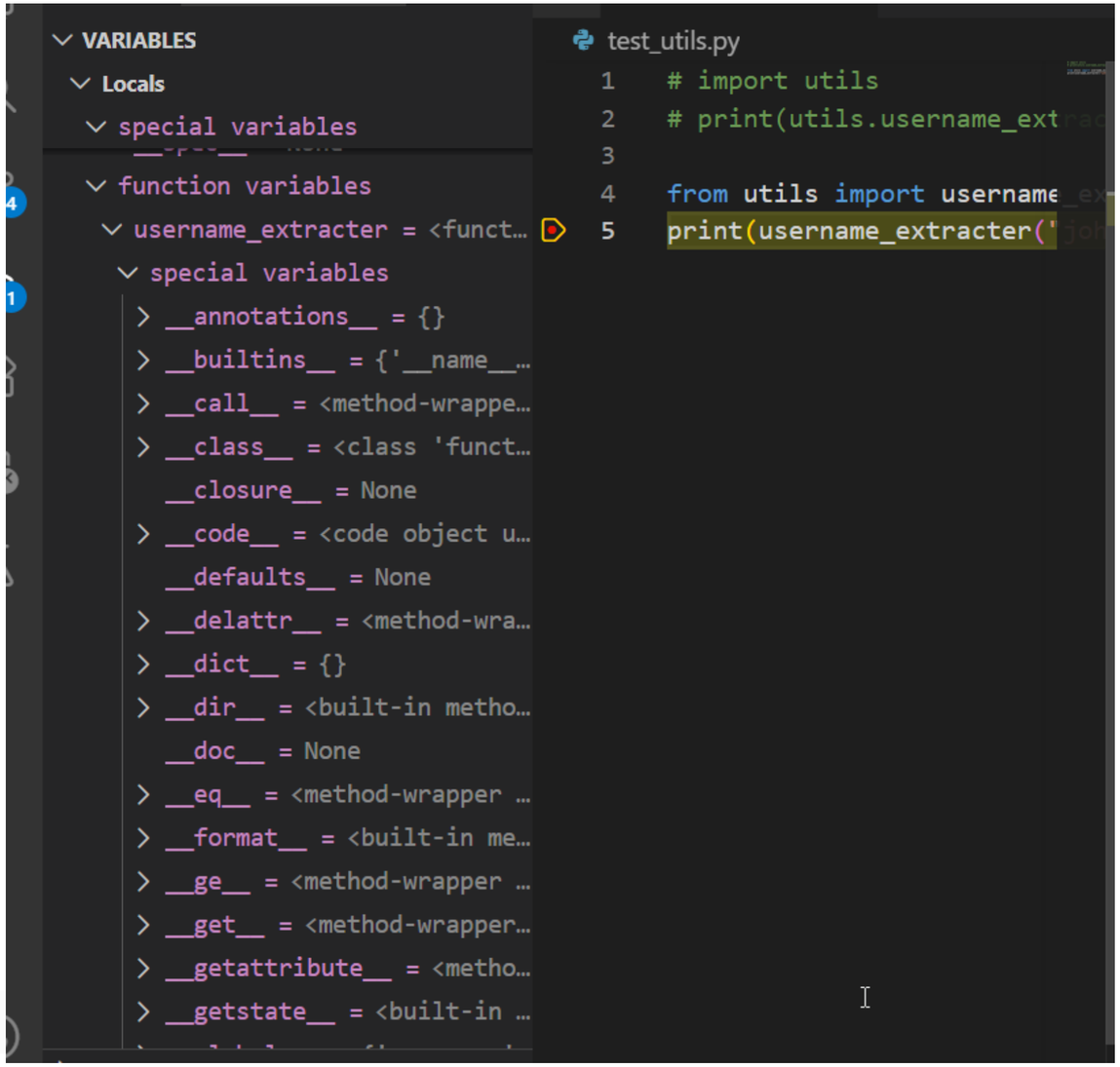
```python
def is_nonlocal_ip(ip_address: str) -> bool:
    '''
    Returns if an IP Address is non-local(public)
    '''
    fields = ip_address.split(".")
    if (fields[0] == "10") or (fields[0] == "172" and int(fields[1]) >= 16 and int(fields[1]) <= 31) or (fi
        return False

    return True

def username_extracter(email: str) -> str:
    '''
    Returns username from email id.
    '''
    return email[:email.find('@')]

def add(a,b):
    return a + b

if __name__ == '__main__':
    uname = username_extracter("vishwas@cloudthat.com")
    print(f"Username is: {uname}")

    print(f"Sum of 23 & 45 is {add(23,45)}")
```

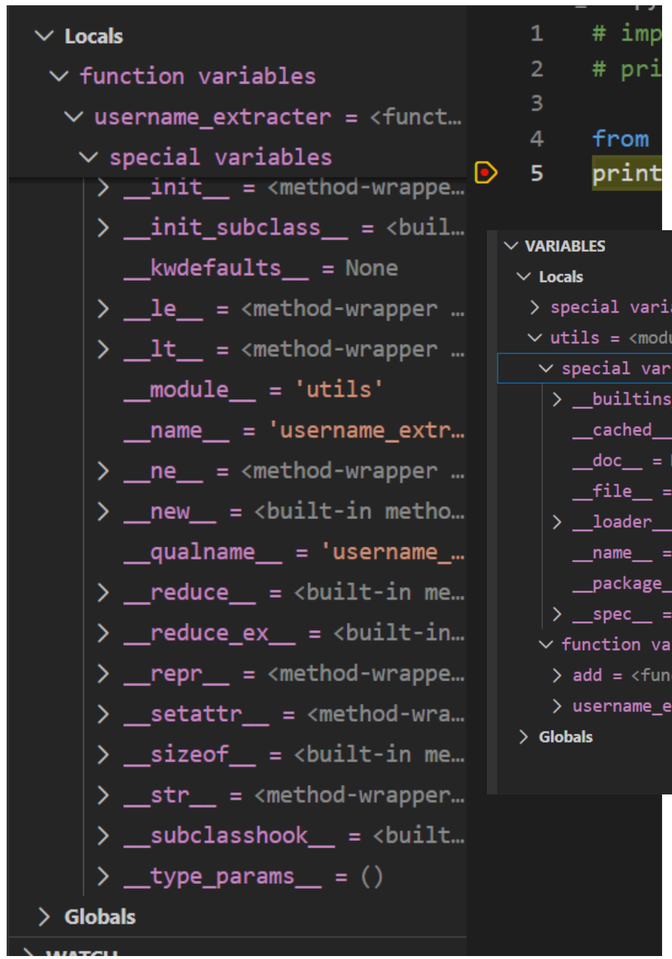
Type Hints

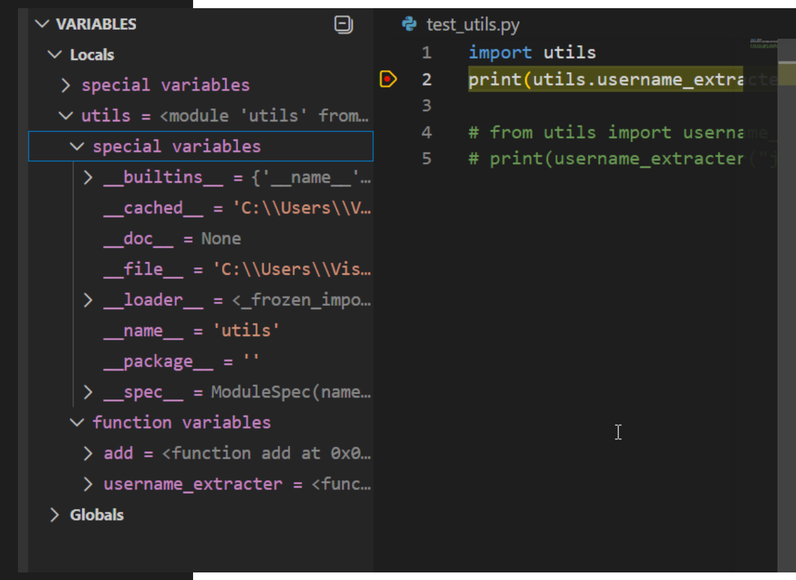
Doc String

Vishwas K Singh
SME-FSD

Creating a module of function definition & calling them

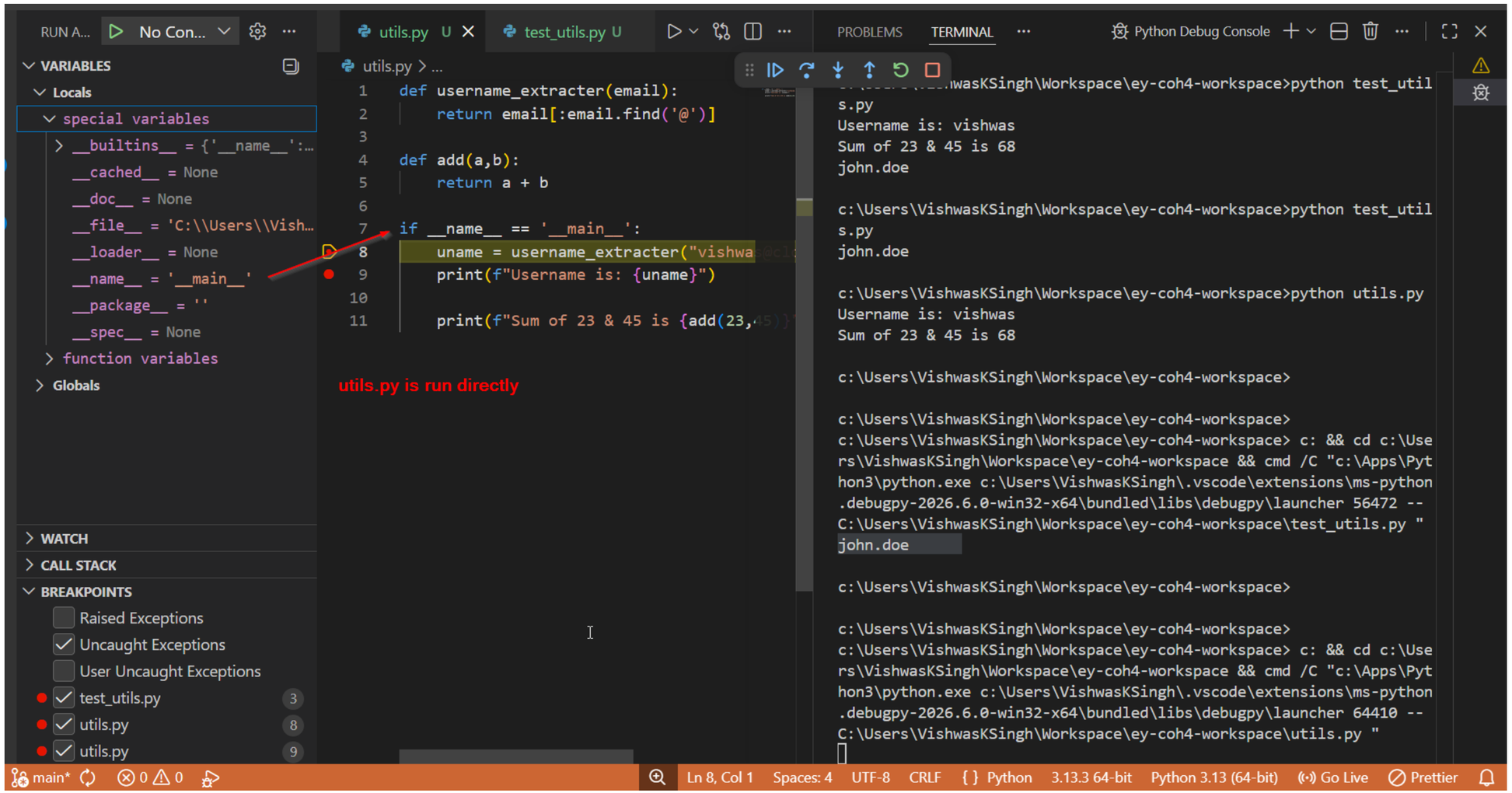






Vishwas K Singh
SME-FSD

Having a Module with function definitions & we want to seperate executions



Vishwas K Singh
SME-FSD

Builtin Modules

https://docs.python.org/3/library/index.html

https://docs.python.org/3/py-modindex.html

Higher Order Functions

- map
- filter
- reduce
- range
- open

Lambda Functions
List Comprehensions
Dict Comprehensions
Iterators
Generators
decorators

```
>>> l1
['12', '34', '56', '78', '9', '10', '23']
>>> l2 = list(map(int,l1))
>>> l2
[12, 34, 56, 78, 9, 10, 23]
>>> type(l2)
<class 'list'>
>>> type(l2[0])
<class 'int'>
>>> help(map)
Help on class map in module builtins:

class map(object)
 |  map(function, iterable, /, *iterables)
 |
 |  Make an iterator that computes the function using arguments from
 |  each of the iterables.  Stops when the shortest iterable is exhausted.
 |
```

```
>>> l3 = list(map(float,input("Enter numbers seperated by spaces: ").split()))
Enter numbers seperated by spaces: 23 45 96 78
>>> l3
[23.0, 45.0, 96.0, 78.0]
>>>
```

Uses of Map Function

Vishwas K Singh
SME-FSD

Uses of Filter Function

```
>>> def iseven(num):
...     return num%2 == 0
...
>>> l1 = [23,45,66,78,92,33,44,498]
>>> even_nums = list(filter(iseven,l1))
>>> even_nums
[66, 78, 92, 44, 498]
>>>
```

Lab: Write a filter function which filters out non-local ip addresses from a list of ip addresses

Practice Problem
given a string validate ipv4 address
1. It has exactly 4 fields seperated by "."
2. each field varies between 0 to 255

Anonymous Functions (lambda functions) - one line, quick functions

```
>>> even_nums = list(filter(lambda x: x%2==0,l1))
>>> even_nums
[66, 78, 92, 44, 498]
>>> square = lambda x: x**2
>>> for i in range(10):
...     print(square(i), end=" ")
...
>>> 4 9 16 25 36 49 64 81
```

Lab: You have a dictionary of usernames & number of failed attempts. Extract the items, usernames which have more than 3 failed attempts

Vishwas K Singh
SME-FSD

List Comprehension

Scenario : Generate a list of 1000 numbers from 0 to 1000

```
l1 = []
for i in range(1000):
    l1.append(i)
```

```
l1 = [x for x in range(1000)]
```

```
l2 = [x for x in range(1000) if x%2 == 0 ]
```

```python
d1 = {
    "vishwas": 9,
    "john": 2,
    "rani": 5,
    "dheeraj": 1
}

print(d1)
# l1 = list(filter(lambda x: x[1]>3, d1.items()))
# Filtering a list using list comprehension
l1 = [(k,v) for k,v in d1.items() if v > 3]
# print(l1)
# d2 = {}
# d2.update(l1)
# print(d2)
# filtering a dictionary using dictionary comprehension
d2 = {k:v for k,v in d1.items() if v > 3}
print(d2)
```

Vishwas K Singh
SME-FSD

Iterators & generators

Scenario: when you read or generate large data everything is stored in RAM your program is using high memory.

How to handle memory efficiently : Iterators & generators

iterator – which gives the value from an iterable only when needed

Generator – function which generates a value when needed, it does not return but yields the value

Generator:

```
>>> def gen_val(n):
...     for i in range(n):
...         yield i
...
>>> for num in gen_val(5):
...     print(num)
...
0
1
2
3
4
>>>
```

iterator:

```
>>> l1 = [1,2,3,4,5,6,7,8,9]
>>> it1 = iter(l1)
>>> next(it1)
1
>>> next(it1)
2
>>> next(it1)
3
>>> next(it1)
4
>>> next(it1)
5
>>> next(it1)
6
>>> next(it1)
7
>>> next(it1)
8
>>> next(it1)
9
>>> next(it1)
Traceback (most recent call last):
  File "<python-input-27>", line 1, in <module>
    next(it1)
    ~~~~^^^^^
StopIteration
>>>
```

Vishwas K Singh
SME-FSD

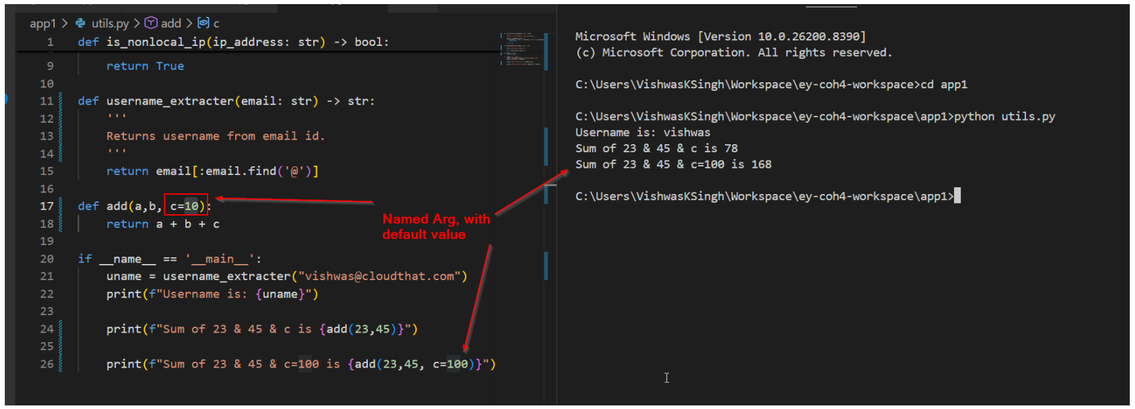
Positional Args

app1 > utils.py > ...

```python
def is_nonlocal_ip(ip_address: str) -> bool:
    return True

def username_extracter(email: str) -> str:
    '''
    Returns username from email id.
    '''
    return email[:email.find('@')]

def add(a,b):
    return a + b

if __name__ == '__main__':
    uname = username_extracter("vishwas@cloudthat.com")
    print(f"Username is: {uname}")

    print(f"Sum of 23 & 45 is {add(23,45)}")
```

Positional Arguments

Keyword Args / Named Args



Explore later

Variable number of Positional & named Arguments

passing the values directly to functions

Vishwas K Singh
SME-FSD

Object Oriented Programming:

What are the 4 Pillars of OOPs
- Abstraction
- Inheritance
- Polymorphism
- Encapsulation

An OOPS Class is a strict template of attributes and methods we can use in a class

1. Agree
2. Not Sure
3. Disagree

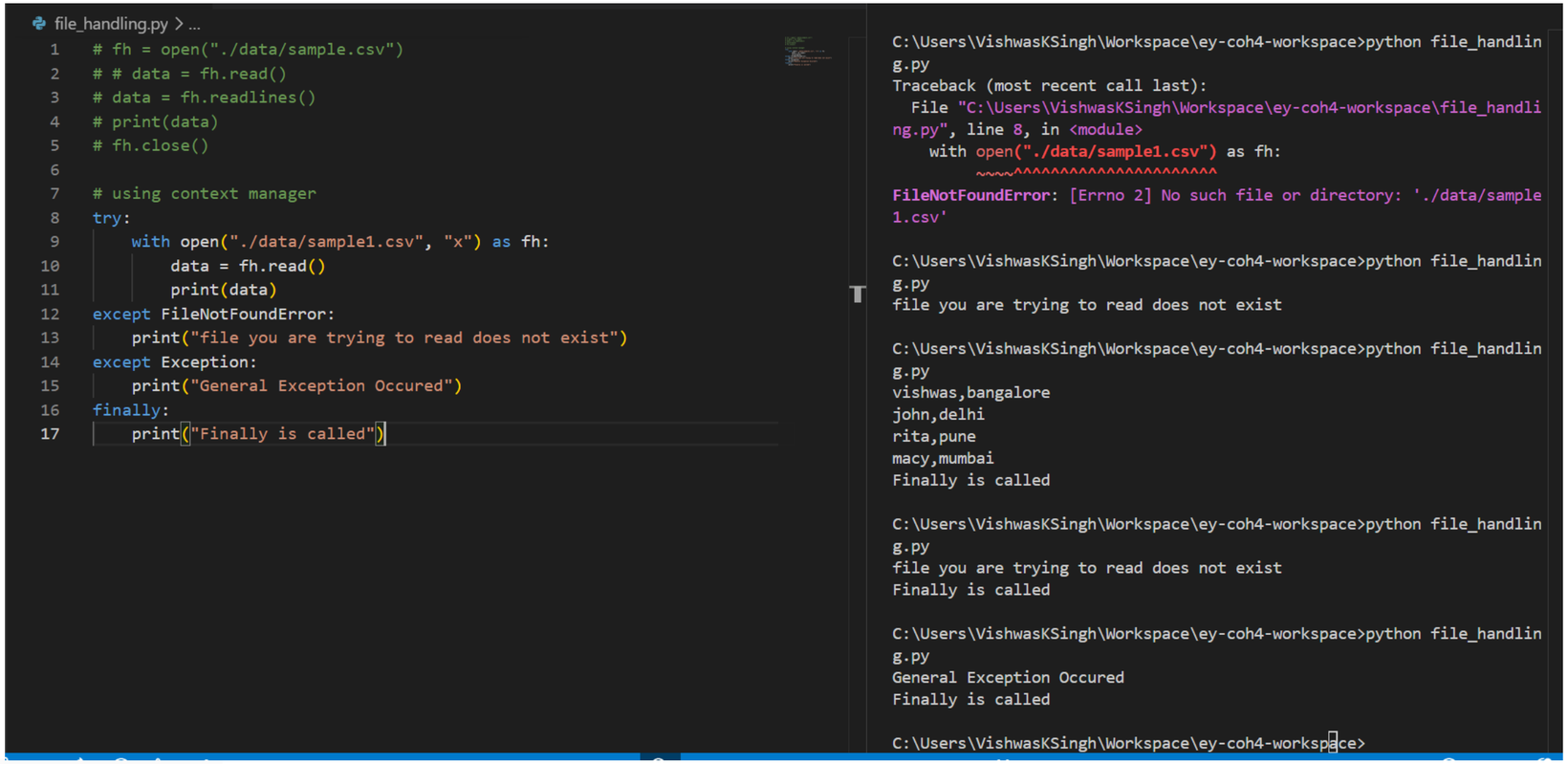
```python
class Person:
    def __init__(self, name, city):
        self.name = name
        self.city = city

    def display_person(self):
        print(f"Name: {self.name}, City: {self.city}")


class User(Person):
    def __init__(self, name, city, salary):
        super().__init__(name, city)
        self.salary = salary

    def display_person(self):
        super().display_person()
        print(f"Salary: {self.salary}")

p1 = Person("vishwas", "Bangalore")
p1.display_person()

p2 = Person("Rita", "Delhi")
p2.display_person()
u1 = User("John", "New York", 20000)
u1.display_person()

p1.location = "India"
print(p1.location)
print(p1.__dict__)
```

```
Traceback (most recent call last):
  File "C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\demo_oops.py", line 39,
in <module>
    print(u1.__b1)
          ^^^^^^^
AttributeError: 'User' object has no attribute '__b1'

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python demo_oops.py
Name: vishwas, City: Bangalore
Traceback (most recent call last):
  File "C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\demo_oops.py", line 35,
in <module>
    print(p1.__b1)
          ^^^^^^^
AttributeError: 'Person' object has no attribute '__b1'

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python demo_oops.py
Name: vishwas, City: Bangalore
Name: Rita, City: Delhi
Name: John, City: New York
Salary: 20000
India

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python demo_oops.py
Name: vishwas, City: Bangalore
Name: Rita, City: Delhi
Name: John, City: New York
Salary: 20000
India
{'name': 'vishwas', 'city': 'Bangalore', 'location': 'India'}

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>
```

main* ⊗ 0 ⚠ 0 — Ln 40, Col 22 — Spaces: 4 — UTF-8 — CRLF — { } Python — 3.13.3 64-bit — Python 3.13 (64-bit) — Go Live — Prettier

File Handling

- Handle - Pointer
- read(single string), readline(set of line), readlines(all the lines as a list)
- by default - read, text



file_handling.py > ...

```
# fh = open("./data/sample.csv")
# # data = fh.read()
# data = fh.readlines()
# print(data)
# fh.close()

# using context manager
try:
    with open("./data/sample1.csv", "x") as fh:
        data = fh.read()
        print(data)
except FileNotFoundError:
    print("file you are trying to read does not exist")
except Exception:
    print("General Exception Occured")
finally:
    print("Finally is called")
```

```
C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python file_handlin
g.py
Traceback (most recent call last):
  File "C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\file_handli
ng.py", line 8, in <module>
    with open("./data/sample1.csv") as fh:
         ~~~~^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: './data/sample
1.csv'

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python file_handlin
g.py
file you are trying to read does not exist

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python file_handlin
g.py
vishwas,bangalore
john,delhi
rita,pune
macy,mumbai
Finally is called

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python file_handlin
g.py
file you are trying to read does not exist
Finally is called

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python file_handlin
g.py
General Exception Occured
Finally is called

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>
```

Vishwas K Singh
SME-FSD

Builtin Modules

pathlib - Manipulate path sanely

Lab: word counter like a program

Take the filename as input from the user
read the file & calculate number of characters, word, line vowels, punctuation digits
write to a sumary.log file in output directory
always the summary to be appended

Input: C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\demo_pathlib.py

File output
Summray of file: demo_pathlib.py

---

number of words: 50
number of digits: 30
number of lines: 10

Vishwas K Singh
SME-FSD

Summary of Day 3

- Functions, Modules, packages
- OOPs Basics
- File & Exception Handling
- Builtin & Custom Modules
- List & Dict Comprehension, Iterators & Generators, Lambda Functions

What to expect Tommorrow
- Third Party Modules - Numpy, Pandas, Matplotlib, Streamlit,
- Capstone
- How to generate PDFs
- How to Create Pipelines

****

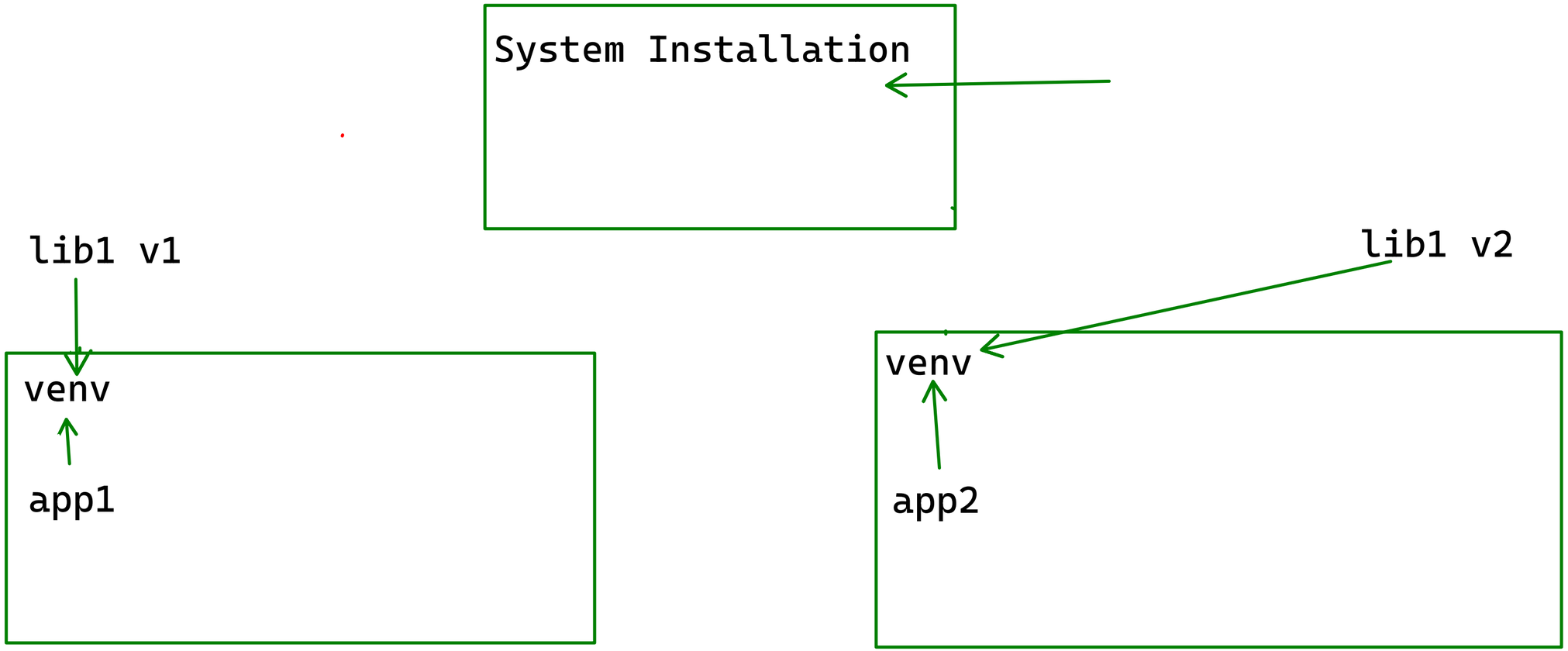
Vishwas K Singh
SME-FSD

Python Training Session Day 4

Agenda
- Third Party Modules - Numpy, Pandas, Matplotlib, Streamlit,
- Capstone
- How to generate PDFs
- How to Create Pipelines

Vishwas K Singh
SME-FSD

Virtual Environment



We will use the module virtual env

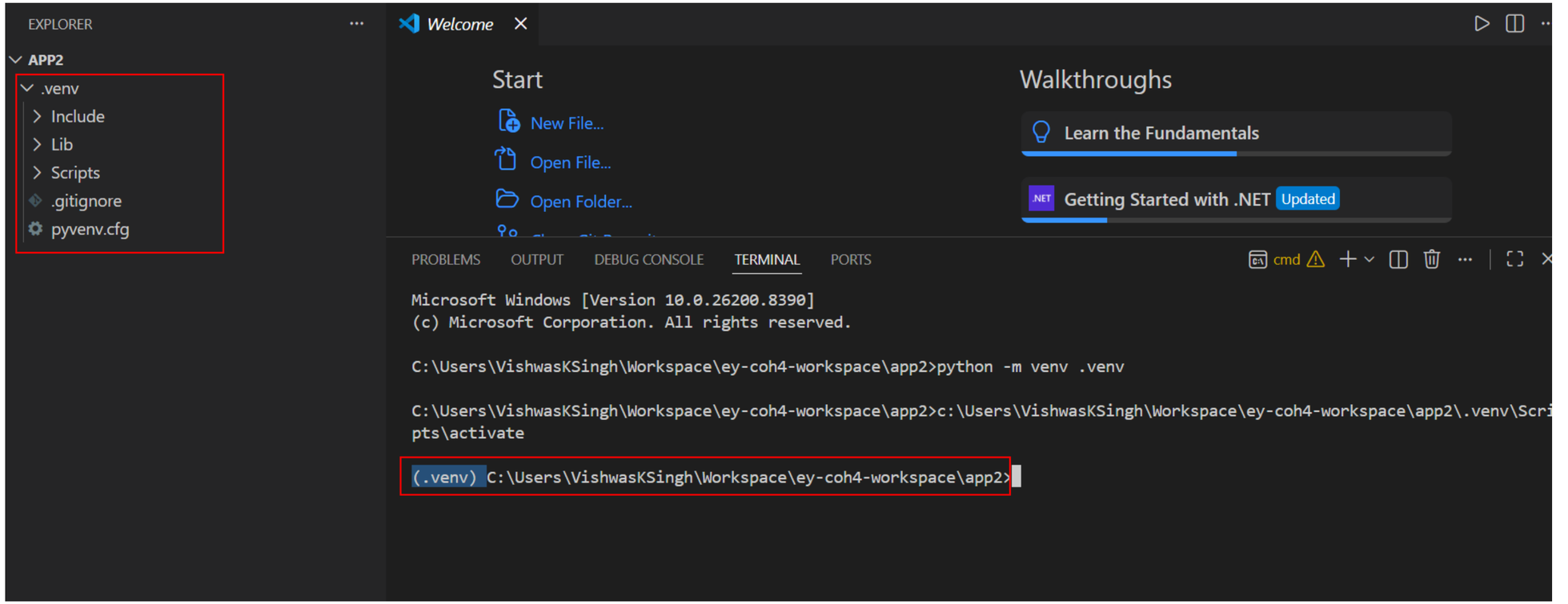
1. `cd` into the folder of application
2. use the command below to create the env
python -m venv .venv

Here `.venv` is the folder where virtual env is created. It can be anything but used as .venv by convention

3. The virtual env is usually deteced by vscode. If not we need to activate
On Windows
.venv\Scripts\lib\Activate.ps1 or Activate.bat
On Linux
source .venv/bin/activate

Vishwas K Singh
SME-FSD

.venv folder is created & activated automatically



pip – tool to manage 3rd party software

```
(.venv) C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\app2>pip install numpy
Collecting numpy
  Using cached numpy-2.4.6-cp313-cp313-win_amd64.whl.metadata (6.6 kB)
Using cached numpy-2.4.6-cp313-cp313-win_amd64.whl (12.3 MB)
Installing collected packages: numpy
Successfully installed numpy-2.4.6

[notice] A new release of pip is available: 25.0.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\app2>pip list
Package Version
------- -------
numpy   2.4.6
pip     25.0.1
```

Vishwas K Singh
SME-FSD

Useful 3rd Party Packages

pydantic - Creating Data Models and Validation of Data
BeautifulSoup - Parsing XML & HTML
NumPy - Doing Matrix Manipulation & Calculation
Scipy - Scientific Computations
Pandas - Python Data Science Library
Matplotlib - Generate Visualization
Seaborn - Generate Visualization compacts/theming etc..
yfinance - Realtime & Historical Stock Price info of NYSE
statistics - Doing Statistical Operations
Sklearn - Machine Learning Models
Keras & Tensorflow - AI Models (KNN, RNN, ...)
HuggingFace - GenAI based AI Models
nltk - Natural Language Transformer Kit
Celery - Performing Heavy Computation Asynchronously
SQLAlchemy - ORM to connect RDBMS Systems
mysqlclient,pgsql - Drivers to connect to RDBMS
mongo - connecting to MongoDB
Streamlit - Interactive Dashboards

Opnpyxl - Working with Excel Files
easy-docx - Working with word documents
Langchain & Langgraph - Workig with AI Agents
OpenCrew

Reportlab, fpdf, Weasyprint - Generating PDF Reports
Python Wrappers - pypandoc - Document Conversion utility

boto3 - AWS
Azure python SDK - Azure

Vishwas K Singh
SME-FSD

Data Validation

Scenario: You have data from a data source (csv/sql/ ...). Right Format

Builtin - DataClasses

Csv record
Name: str,City:str,Salary:float

case 1: "vishwas","Pune",20000
case 2: "john","Delhi","123"

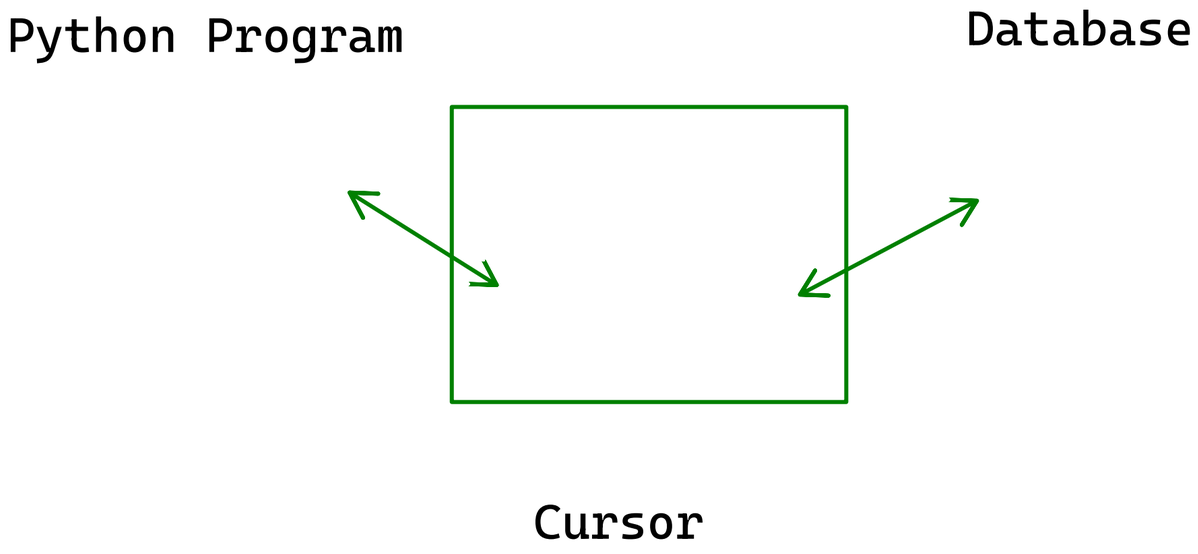
Vishwas K Singh
SME-FSD

Connecting to the database

Analysis using SQL Statements
Store the data or perform CRUD Operations

use sqlite3

Driver - mysqlclient, pgsql.....
SQLAlchemy - Pythonic way
Pandas - Doing analysis & store & retrieve data

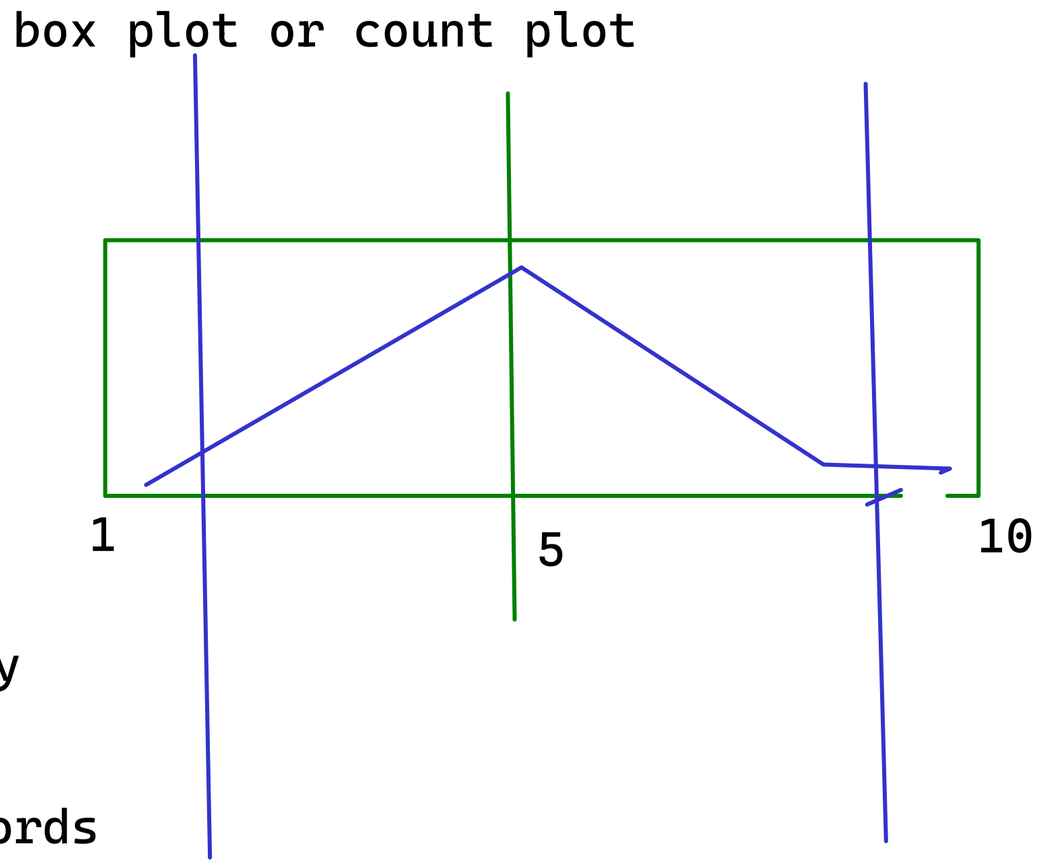


Vishwas K Singh
SME-FSD

Pandas Library

- many data formats

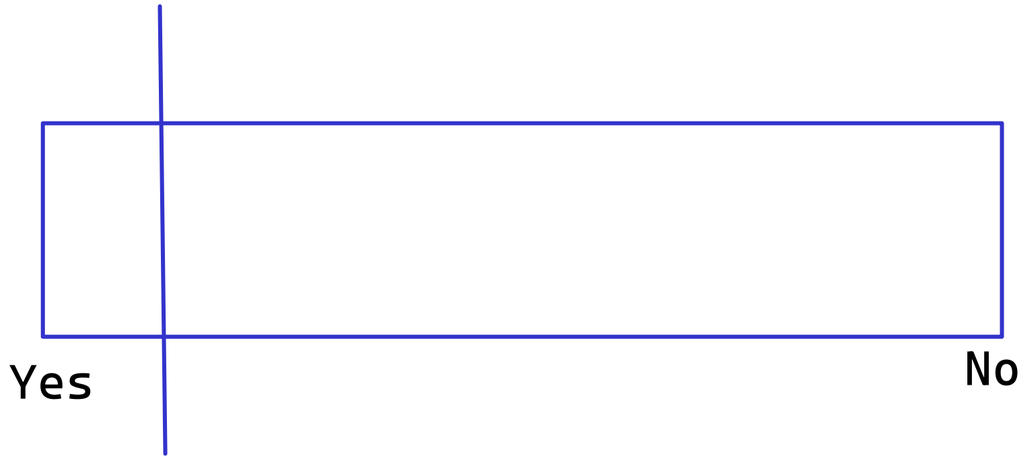
2 Basic Data Structures
- Series - 1D, Homogeneous Data
- DataFrame - 2D, Heterogeneous Data



If I have some data which varies between 1 and 10. Idealy a balanced data should have equal distribution in the range such that ideally majority of records fall around mean

But sometimes the data might be skewed (more records fall in one extreme or some records which are too extremes from the edges)

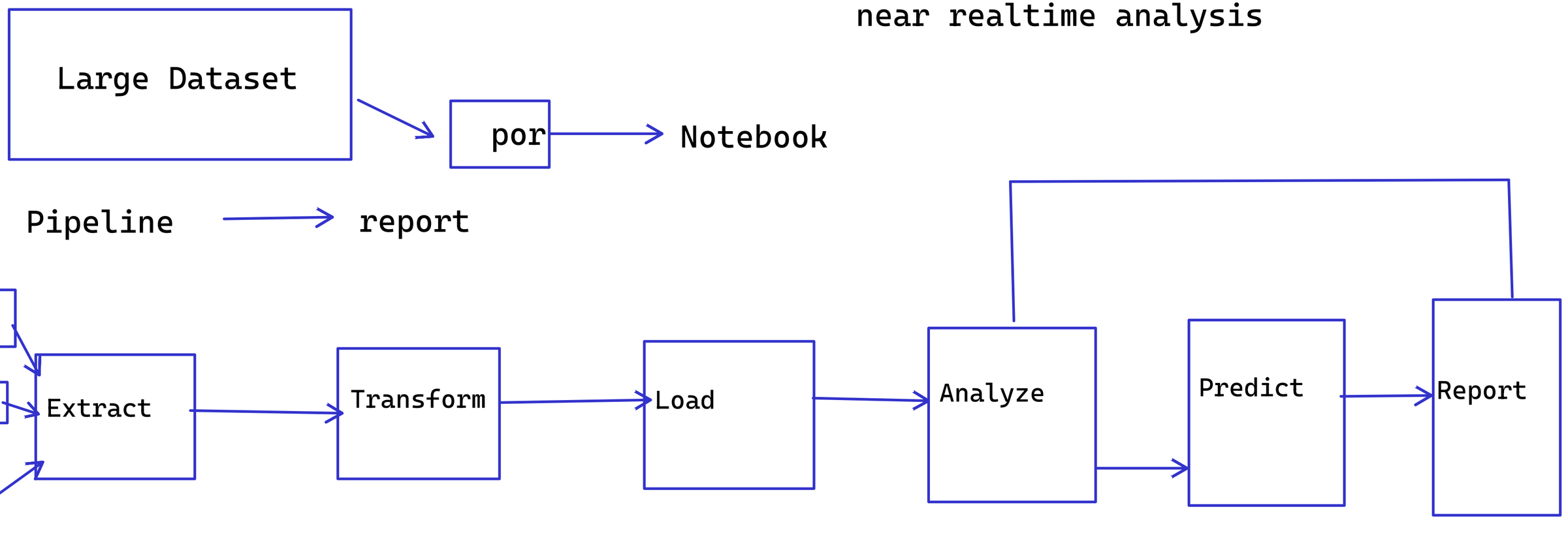
This makes our analysis & predictions baised towards that edge



The prediction results will be more towards yes

Vishwas K Singh
SME-FSD

near realtime analysis



Vishwas K Singh

SME-FSD

Summary Day 4

- Various third party Modules

Where to go from here
- Faced with a task, think how to automate
- Explore ML/AI Models
- Explore AI Agents & toolkits

****

Email: vishwas@cloudthat.com